

Software Project Management

Software project management is the process of planning, organizing, leading, and controlling software projects. It involves the application of knowledge, skills, tools, and techniques to achieve specific objectives within the constraints of time, budget, scope, and quality.

Some of the topics that are covered in software project management are:

- Software project life cycle: The phases and activities that a software project goes through from initiation to closure. The common models are waterfall, iterative, incremental, agile, and hybrid.
- Software project planning: The process of defining the scope, objectives, deliverables, schedule, resources, risks, and quality standards of a software project. It also involves estimating the cost, effort, and duration of the project, and creating a project plan document.
- Software project execution: The process of implementing the project plan, performing the tasks, managing the resources, monitoring the progress, and communicating the status of a software project. It also involves managing changes, issues, and risks that may arise during the project.
- Software project control: The process of measuring the performance of a software project against the planned baselines, and taking corrective actions if there are any deviations or variances. It also involves reporting the project performance to the stakeholders, and ensuring the quality of the project deliverables.
- Software project closure: The process of finalizing and delivering the project deliverables, verifying the customer satisfaction, releasing the project resources, and documenting the project lessons learned. It also involves conducting a project review and evaluation, and celebrating the project success.

Some of the tools and techniques that are used in software project management are:

- Work breakdown structure (WBS): A hierarchical decomposition of the project scope into smaller and manageable units called work packages. It helps to define the project deliverables, assign responsibilities, and estimate the cost and effort of the project.
- Gantt chart: A graphical representation of the project schedule, showing the start and end dates, durations, dependencies, and milestones of the project tasks. It helps to visualize the project timeline, track the progress, and identify the critical path of the project.
- Network diagram: A graphical representation of the project tasks and their logical relationships, showing the sequence, precedence, and concurrency of the project activities. It helps to analyze the project dependencies, estimate the project duration, and optimize the project schedule.
- Resource histogram: A graphical representation of the project resources and their allocation over time, showing the resource availability, demand,

and utilization. It helps to plan the project resources, balance the resource workload, and resolve the resource conflicts.

- Risk register: A document that records the project risks and their characteristics, such as probability, impact, priority, response strategy, and status. It helps to identify, analyze, prioritize, and manage the project risks.
- Earned value management (EVM): A technique that integrates the project scope, schedule, and cost, and measures the project performance using three key metrics: planned value (PV), earned value (EV), and actual cost (AC). It helps to calculate the project variance, performance index, and forecast.
- Quality management plan: A document that describes the quality standards, criteria, methods, and tools that will be used to ensure the quality of the project deliverables and processes. It helps to define the quality expectations, measure the quality performance, and implement the quality improvement actions.

Unit - 1 - Project Evaluation and Project Planning for the notes of the course Software Project Management

- Project Evaluation and Project Planning are two important aspects of Software Project Management, which is the discipline of defining and achieving targets while optimizing the use of resources (time, money, people, materials, energy, space, etc) over the course of a project (a set of activities of finite duration and scope) .
- Project Evaluation is the process of assessing the feasibility, desirability, and viability of a project before, during, and after its execution. Project Evaluation helps to determine the project's objectives, scope, risks, costs, benefits, and outcomes, and to compare them with the expectations and requirements of the stakeholders .
- Project Planning is the process of defining the project's activities, tasks, milestones, deliverables, resources, schedule, and budget, and of establishing the project's management and control mechanisms. Project Planning helps to guide the project's execution, monitoring, and closure, and to ensure that the project meets its objectives and delivers value to the stakeholders .
- Some of the topics covered in this unit are:
 - Importance of Software Project Management: Software Project Management is important because software projects are often complex, uncertain, dynamic, and unique, and require a systematic and disciplined approach to ensure their success and quality. Software Project Management helps to align the software development process with the business goals and customer needs, and to manage the trade-offs between the project's scope, time, cost, and quality .

- Activities of Software Project Management: Software Project Management involves various activities, such as:
 - * Initiation: defining the project's purpose, scope, objectives, and stakeholders, and obtaining their approval and commitment.
 - * Planning: defining the project's activities, tasks, milestones, deliverables, resources, schedule, and budget, and establishing the project's management and control mechanisms.
 - * Execution: performing the project's activities and tasks, producing the project's deliverables, and managing the project's resources, risks, issues, and changes.
 - * Monitoring and Control: measuring the project's performance and progress, comparing them with the project's plan, and taking corrective actions if necessary.
 - * Closure: completing the project's activities and tasks, delivering the project's final product, and evaluating the project's outcomes and lessons learned .
- Methodologies of Software Project Management: Software Project Management can follow different methodologies, which are frameworks or guidelines that prescribe how to plan, execute, monitor, and control a software project. Some of the common methodologies are:
 - * Waterfall: a sequential and linear approach that divides the software development process into distinct phases, such as requirements analysis, design, implementation, testing, and maintenance, and requires each phase to be completed and verified before moving to the next one.
 - * Agile: an iterative and incremental approach that divides the software development process into short cycles, called sprints, and emphasizes collaboration, communication, feedback, and adaptation to changing requirements and customer needs.
 - * Scrum: a specific type of agile methodology that uses a self-organizing and cross-functional team, a product owner, and a scrum master, and follows a set of roles, events, artifacts, and rules to deliver software products in sprints.
 - * Kanban: a specific type of agile methodology that uses a visual system, such as a board or a card, to represent the workflow of the software development process, and follows a set of principles and practices to optimize the flow of work and minimize waste .
- Categorization of Software Projects: Software Projects can be categorized based on different criteria, such as:
 - * Size: the amount of effort, time, and resources required to complete the project, which can be measured by metrics such as lines of code, function points, or person-months.
 - * Complexity: the degree of difficulty, uncertainty, and interdependence involved in the project, which can be influenced by factors such as the project's scope, requirements, technology, ar-

chitecture, and stakeholders.

- * Type: the nature and purpose of the software product to be developed, which can be classified by domains such as web, mobile, embedded, enterprise, or scientific applications.
- * Lifecycle: the stage or phase of the software development process that the project is in, which can range from inception, to development, to deployment, to maintenance, to retirement [3]

Importance of Software Project Management

Software project management is the process of planning, organizing, leading, and controlling software projects. It involves activities such as defining the scope, objectives, and deliverables of the project, estimating the resources, time, and cost required, assigning tasks and roles to the project team, monitoring and controlling the progress and quality of the project, and communicating with the stakeholders and customers.

Some of the importance of software project management are:

- It helps to ensure that the software project meets the customer's needs and expectations, and delivers the desired value and benefits.
- It helps to manage the risks and uncertainties associated with software development, such as changing requirements, technical challenges, human factors, and external dependencies.
- It helps to optimize the use of the available resources, such as time, money, people, and tools, and to avoid waste and inefficiency.
- It helps to coordinate and integrate the efforts of the project team, and to foster collaboration and communication among the team members and other stakeholders.
- It helps to improve the quality and reliability of the software product, and to ensure that it conforms to the standards and regulations.
- It helps to measure and evaluate the performance and outcomes of the project, and to identify and implement the lessons learned and best practices for future projects.

Some of the mnemonics and learning tricks for the importance of software project management are:

- You can use the acronym SMART to remember the five main characteristics of a good software project: Specific, Measurable, Achievable, Relevant, and Time-bound.
- You can use the acronym RACI to remember the four main types of roles and responsibilities in a software project: Responsible, Accountable, Consulted, and Informed.
- You can use the acronym PDCA to remember the four main phases of a software project: Plan, Do, Check, and Act.

Activities in Software Project Management

Software project management is the process of planning, organizing, leading, and controlling software projects. It involves a range of activities that help software teams achieve their goals and deliver high-quality software that meets the needs of the clients. Some of the main activities in software project management are:

- **Scope management:** This activity involves defining and controlling the scope of the software project, which includes the features, functions, and requirements of the software product. Scope management helps to avoid scope creep, which is the tendency of the project to expand beyond its original objectives. Scope management also helps to ensure that the project delivers the expected value to the stakeholders.
- **Project planning and tracking:** This activity involves creating and maintaining a project plan, which is a document that describes the objectives, tasks, milestones, deliverables, resources, and schedule of the software project. Project planning and tracking helps to coordinate the work of the software team, monitor the progress and performance of the project, and identify and resolve any issues or risks that may arise. Project planning and tracking also helps to communicate the project status and expectations to the stakeholders.
- **Project resource management:** This activity involves managing the human, physical, and financial resources of the software project. Project resource management helps to allocate and optimize the use of the resources, such as the software team members, equipment, tools, and budget. Project resource management also helps to motivate and develop the skills of the software team, and to manage any conflicts or changes that may affect the resources.
- **Scheduling management:** This activity involves creating and updating a project schedule, which is a timeline that shows the start and end dates of the tasks and milestones of the software project. Scheduling management helps to estimate the duration and effort of the tasks, assign the tasks to the software team members, and track the completion and dependencies of the tasks. Scheduling management also helps to manage any delays or changes that may impact the project schedule.
- **Project communication management:** This activity involves planning and executing the communication activities of the software project, such as meetings, reports, presentations, and feedback. Project communication management helps to ensure that the software team and the stakeholders are well-informed, engaged, and aligned on the project goals, scope, status, and issues. Project communication management also helps to build trust and rapport among the software team and the stakeholders, and to manage any expectations or conflicts that may arise.
- **Estimation management:** This activity involves estimating the cost,

time, and quality of the software project, based on the scope, requirements, and resources of the project. Estimation management helps to set realistic and achievable goals for the software project, and to measure the return on investment (ROI) of the project. Estimation management also helps to manage any uncertainties or risks that may affect the project estimates.

- **Project risk management:** This activity involves identifying, analyzing, and mitigating the potential risks that may threaten the success of the software project. Risks are any events or conditions that may have a negative impact on the project objectives, scope, schedule, budget, or quality. Project risk management helps to prevent or reduce the likelihood and impact of the risks, and to prepare contingency plans for the risks. Project risk management also helps to monitor and control the risks throughout the project lifecycle.
- **Project configuration management:** This activity involves managing the changes and versions of the software product and the project artifacts, such as the requirements, design, code, test cases, and documentation. Project configuration management helps to ensure the consistency, integrity, and traceability of the software product and the project artifacts, and to prevent any errors or conflicts that may arise from the changes. Project configuration management also helps to facilitate the collaboration and integration of the software team and the stakeholders.

These are some of the main activities in software project management, but there may be other activities depending on the specific context, methodology, and scope of the software project. Software project management is a complex and dynamic process that requires a lot of skills, knowledge, and experience from the software project manager and the software team. Software project management is also a key factor for the success and quality of the software product and the satisfaction of the clients.

Methodologies in Software Project Management

- Software project management is the process of planning, organizing, leading, and controlling software projects.
- Software project management methodologies are frameworks or guidelines that help software project managers and teams to deliver software products or services that meet the customer's needs and expectations, within the given constraints of time, budget, and quality.
- There are different types of software project management methodologies, each with its own strengths, weaknesses, and suitability for different kinds of projects. Some of the most common ones are:
 - **Waterfall model:** This is a linear and sequential approach, where the project is divided into distinct phases, such as requirements analysis, design, implementation, testing, and deployment. Each phase

must be completed and verified before moving on to the next one. This model is simple, easy to understand, and suitable for projects with clear and stable requirements, but it is rigid, inflexible, and does not accommodate changes or feedback well.

- **Agile methods:** These are iterative and incremental approaches, where the project is divided into small and manageable units of work, called iterations or sprints, that deliver working software or features to the customer frequently. Each iteration involves planning, designing, developing, testing, and reviewing the software, and the customer's feedback is incorporated into the next iteration. This model is flexible, adaptive, and responsive to changes and customer needs, but it requires high collaboration, communication, and discipline among the project team and stakeholders, and it may not be suitable for projects with complex or fixed requirements, or projects that require extensive documentation or formal processes.
- **Spiral model:** This is a risk-driven approach, where the project is divided into cycles, each consisting of four phases: planning, risk analysis, engineering, and evaluation. In each cycle, the project team identifies the objectives, alternatives, and constraints of the project, analyzes the potential risks and their mitigation strategies, develops and tests the software, and evaluates the results and feedback. The project team can then decide whether to proceed to the next cycle, or terminate the project. This model is suitable for projects with high uncertainty, complexity, or risk, but it requires a lot of expertise, resources, and management to perform risk analysis and evaluation effectively.
- **Rational Unified Process (RUP):** This is a disciplined and structured approach, based on the best practices of software engineering, such as iterative development, requirements management, component-based architecture, visual modeling, quality assurance, and change management. The project is divided into four phases: inception, elaboration, construction, and transition, each consisting of multiple iterations. In each iteration, the project team follows six core engineering workflows: business modeling, requirements, analysis and design, implementation, testing, and deployment, and three supporting workflows: project management, configuration and change management, and environment. This model is comprehensive, customizable, and suitable for large and complex projects, but it requires a lot of effort, documentation, and tools to implement and follow the process effectively.

Categorization of Software Projects in spm

- Software projects can be categorized based on different criteria, such as size, complexity, domain, methodology, or quality attributes.
- One common way of categorizing software projects is based on the **ap-**

application domain, which refers to the area of knowledge or activity for which the software is intended. Some examples of application domains are:

- Business software: software that supports the operations and management of a business, such as accounting, inventory, payroll, etc.
 - Scientific software: software that performs scientific calculations, simulations, or experiments, such as physics, chemistry, biology, etc.
 - Engineering software: software that supports the design, analysis, or testing of engineering systems, such as mechanical, electrical, civil, etc.
 - Embedded software: software that runs on a dedicated hardware device, such as a microcontroller, sensor, or actuator, and performs a specific function, such as controlling a car engine, a robot, or a medical device.
 - Web software: software that runs on a web server or a web browser, and provides information or services over the internet, such as e-commerce, social media, online games, etc.
 - Mobile software: software that runs on a mobile device, such as a smartphone, tablet, or smartwatch, and provides functionality or entertainment, such as messaging, navigation, fitness, etc.
- Another common way of categorizing software projects is based on the **development methodology**, which refers to the process or framework that guides the planning, execution, and delivery of the software. Some examples of development methodologies are:
 - Waterfall: a linear and sequential approach, where the software is developed in phases, such as requirements, design, implementation, testing, and deployment, and each phase is completed before moving to the next one.
 - Agile: an iterative and incremental approach, where the software is developed in short cycles, called sprints, and each sprint delivers a working product or feature, which is then evaluated and improved based on feedback from the customer or the user.
 - Spiral: a risk-driven approach, where the software is developed in loops, called spirals, and each spiral consists of four stages: planning, risk analysis, engineering, and evaluation, and the level of detail and complexity increases with each spiral.
 - Prototyping: an exploratory approach, where the software is developed in stages, called prototypes, and each prototype is a partial or simplified version of the software, which is then tested and refined based on feedback from the customer or the user.
 - Hybrid: a combination of two or more methodologies, where the software is developed using the best practices or techniques from different approaches, depending on the context and the needs of the project.
 - A third way of categorizing software projects is based on the **quality attributes**, which refer to the non-functional characteristics or properties

of the software, such as performance, reliability, security, usability, etc. Some examples of quality attributes are:

- Performance: the degree to which the software meets the speed, efficiency, or throughput requirements, such as response time, resource consumption, or scalability.
- Reliability: the degree to which the software meets the availability, fault tolerance, or recoverability requirements, such as uptime, error handling, or backup.
- Security: the degree to which the software meets the confidentiality, integrity, or authenticity requirements, such as encryption, access control, or authentication.
- Usability: the degree to which the software meets the ease of use, learnability, or satisfaction requirements, such as user interface, user feedback, or user experience.
- Maintainability: the degree to which the software meets the modifiability, testability, or reusability requirements, such as code quality, documentation, or design patterns.

Setting Objectives in spm

- Setting objectives in software project management (spm) is the process of defining and prioritizing the desired outcomes and benefits of a software project.
- Objectives provide a clear direction and focus for the project team and stakeholders, and help to align the project activities with the organizational strategy and vision.
- Objectives should be SMART: Specific, Measurable, Achievable, Relevant, and Time-bound.
- Specific: Objectives should state clearly what is to be accomplished, by whom, where, and when. They should avoid vague or ambiguous terms and use precise language.
- Measurable: Objectives should have quantifiable indicators or criteria that can be used to track and evaluate the progress and success of the project. They should define the expected quality, quantity, cost, and time of the deliverables and outcomes.
- Achievable: Objectives should be realistic and attainable, considering the available resources, constraints, risks, and dependencies of the project. They should not be too easy or too hard, but challenging enough to motivate the project team and stakeholders.
- Relevant: Objectives should be aligned with the project scope, vision, and purpose, and support the strategic goals and needs of the organization and the customers. They should also be consistent with the project values and principles, and reflect the interests and expectations of the stakeholders.

- Time-bound: Objectives should have a clear and realistic deadline or time-frame for completion. They should also specify the milestones and deliverables that mark the progress and completion of the project.
- A mnemonic to remember the SMART criteria is: **See My Awesome Results Today**.
- A technique to set objectives in spm is the SMARTER framework, which adds two additional criteria: Evaluate and Review.
- Evaluate: Objectives should be assessed and measured regularly throughout the project lifecycle, using the predefined indicators and criteria. This helps to monitor the project performance, identify and resolve issues, and ensure the quality and value of the project outcomes and benefits.
- Review: Objectives should be reviewed and revised periodically, based on the feedback and lessons learned from the evaluation process. This helps to adapt to the changing requirements, expectations, and environment of the project, and ensure the relevance and alignment of the project objectives.
- A mnemonic to remember the SMARTER criteria is: **See My Awesome Results Today, Everyone Raves**.

Management Principles in SPM

SPM stands for Software Project Management or Strategic Portfolio Management, depending on the context. Both are related to managing the resources, processes, and outcomes of projects or portfolios that involve software development or innovation. Here are some of the common management principles in SPM:

- **Define and document requirements:** A clear and detailed specification of the project or portfolio scope, objectives, deliverables, constraints, assumptions, and risks is essential for effective planning, execution, and evaluation of SPM.
- **Manage changes effectively:** Changes are inevitable in any project or portfolio, especially in dynamic and uncertain environments. SPM requires a systematic and flexible approach to identify, assess, approve, communicate, and implement changes, while minimizing the negative impacts and maximizing the positive opportunities.
- **Prioritize projects:** SPM involves managing multiple projects or portfolios that may have different goals, stakeholders, resources, and risks. SPM requires a rational and transparent method to rank and select the most valuable and feasible projects or portfolios, based on their alignment with the strategic vision, mission, and objectives of the organization .
- **Maintain quality:** Quality is the degree to which the project or portfolio meets or exceeds the expectations of the customers, users, and stakeholders. SPM requires a continuous and proactive process to define, measure, monitor, and improve the quality of the project or portfolio outputs and

outcomes, as well as the quality of the processes and practices involved in SPM.

- **Utilize automation:** Automation is the use of technology to perform tasks or processes that would otherwise require human intervention or effort. SPM can benefit from automation in various aspects, such as data collection, analysis, reporting, communication, testing, deployment, and maintenance. Automation can enhance the efficiency, accuracy, consistency, and scalability of SPM.
- **Establish communication protocols:** Communication is the exchange of information, ideas, feedback, and decisions among the project or portfolio team members, stakeholders, and other parties involved in SPM. SPM requires a clear and consistent communication plan that specifies the purpose, frequency, mode, format, and content of communication, as well as the roles and responsibilities of the communicators and the recipients.
- **Manage risk:** Risk is the uncertainty or variability of the project or portfolio outcomes, which can have positive or negative effects on the project or portfolio objectives. SPM requires a systematic and proactive process to identify, analyze, evaluate, mitigate, monitor, and control the risks that may affect the project or portfolio performance, quality, scope, schedule, and budget.
- **Utilize a systematic approach:** A systematic approach is a structured and logical method to plan, execute, and evaluate the project or portfolio activities and results. SPM requires a systematic approach that follows a standard or customized framework, model, or methodology that guides the project or portfolio lifecycle, from initiation to closure, and ensures the alignment, integration, and coherence of the project or portfolio components .

: <https://www.gristprojectmanagement.us/software-3/the-principles-of-modern-software-manag>
<https://www.geeksforgeeks.org/software-engineering-software-project-management-spm/>
<https://www.servicenow.com/products/strategic-portfolio-management/what-is-spm.html>
<https://smartorg.com/resources/principles-strategic-portfolio-management/>

Management Control in spm

Management control in software project management (spm) is the process of monitoring and controlling project activities to ensure the project meets its goals and stays on budget. It involves setting performance targets, measuring progress towards those targets, and taking corrective action to ensure the project is on track.

Some of the key aspects of management control in spm are:

- **Scope control:** This is the process of defining and managing the scope of the project, which includes the features and functions of the software

product, the deliverables, the assumptions, and the constraints. Scope control helps to avoid scope creep, which is the tendency of the project to expand beyond its original objectives.

- **Schedule control:** This is the process of developing and maintaining a realistic and feasible project schedule, which defines the sequence and duration of project tasks and milestones. Schedule control helps to avoid schedule slippage, which is the delay in completing the project on time.
- **Cost control:** This is the process of estimating and managing the budget of the project, which includes the resources, materials, and labor required to complete the project. Cost control helps to avoid cost overrun, which is the excess of actual project cost over the estimated project cost.
- **Quality control:** This is the process of ensuring that the software product meets the quality standards and expectations of the stakeholders, which includes the functional and non-functional requirements, the design specifications, and the testing criteria. Quality control helps to avoid quality defects, which are the errors or faults in the software product that affect its performance or functionality.
- **Risk control:** This is the process of identifying and managing the potential threats and opportunities that may affect the project, which includes the technical, organizational, environmental, and external factors. Risk control helps to avoid or mitigate the negative impacts of risks and to exploit or enhance the positive impacts of opportunities.

Some of the tools and techniques that can be used for management control in spm are:

- **Project plan:** This is the document that describes the scope, schedule, cost, quality, and risk of the project, as well as the roles and responsibilities of the project team, the communication and reporting mechanisms, and the change and issue management procedures. The project plan serves as the baseline for measuring and controlling the project performance.
- **Work breakdown structure (WBS):** This is the hierarchical decomposition of the project scope into smaller and manageable units, which are called work packages. Each work package has a unique identifier, a description, a duration, a cost, and a set of deliverables. The WBS helps to define and organize the project work and to allocate the resources and responsibilities.
- **Gantt chart:** This is a graphical representation of the project schedule, which shows the start and end dates of the project tasks and milestones, as well as the dependencies and constraints among them. The Gantt chart helps to visualize and track the project progress and to identify and resolve any schedule conflicts or delays.
- **Earned value management (EVM):** This is a method of measuring and reporting the project performance, which integrates the scope, schedule, and cost aspects of the project. EVM uses three key metrics: the planned value (PV), which is the budgeted cost of the work scheduled; the earned value (EV), which is the budgeted cost of the work completed; and

the actual cost (AC), which is the actual cost of the work completed. EVM also calculates three key indicators: the schedule variance (SV), which is the difference between EV and PV; the cost variance (CV), which is the difference between EV and AC; and the schedule performance index (SPI) and the cost performance index (CPI), which are the ratios of EV to PV and EV to AC, respectively. EVM helps to evaluate the project status and to forecast the project outcome.

- **Quality assurance (QA) and quality control (QC):** These are the processes of ensuring and verifying the quality of the software product, respectively. QA involves the planning and implementation of the quality standards and procedures, such as the quality management plan, the quality audit, and the quality review. QC involves the testing and inspection of the software product, such as the unit testing, the integration testing, the system testing, and the user acceptance testing. QA and QC help to identify and correct any quality defects and to ensure the customer satisfaction.
- **Risk register:** This is the document that records the identified risks and their characteristics, such as the probability, the impact, the priority, the trigger, the owner, and the response strategy. The risk register helps to monitor and control the risks and to update the risk status and information.

Risk Evaluation in spm

- Risk evaluation is the process of assessing the likelihood and impact of potential risks in a software project management (spm) context.
- Risk evaluation helps to prioritize the risks and decide on the appropriate risk response strategies.
- Risk evaluation can be done qualitatively or quantitatively, depending on the availability and quality of data, the complexity and uncertainty of the project, and the stakeholder preferences.
- Qualitative risk evaluation involves assigning a rating to each risk based on its probability and severity, using a predefined scale or matrix. For example, a risk can be rated as low, medium, or high, or as 1 to 5, where 1 is the lowest and 5 is the highest.
- Quantitative risk evaluation involves estimating the numerical values of the probability and impact of each risk, using statistical methods, models, or simulations. For example, a risk can be estimated as having a 20% chance of occurring and causing a \$10,000 loss.
- The output of risk evaluation is a prioritized list of risks, with their ratings or values, that can be used to plan the risk response actions.
- Some of the benefits of risk evaluation are:
 - It helps to identify the most critical risks that need immediate attention and mitigation.
 - It helps to allocate the resources and budget for risk management effectively and efficiently.

- It helps to communicate the risk information to the stakeholders and gain their support and commitment.
- It helps to monitor and control the risk performance and adjust the risk response plans as needed.
- Some of the challenges of risk evaluation are:
 - It can be subjective and biased, depending on the judgment and experience of the risk evaluators and the quality of the data.
 - It can be time-consuming and costly, especially for quantitative methods that require sophisticated tools and techniques.
 - It can be uncertain and inaccurate, due to the inherent variability and unpredictability of the risk factors and events.
- Some of the best practices for risk evaluation are:
 - Define the criteria and scales for risk rating or valuation, and ensure they are consistent and agreed upon by the stakeholders.
 - Use multiple sources and methods of data collection and analysis, and validate the results with experts and historical data.
 - Involve the relevant stakeholders and experts in the risk evaluation process, and solicit their feedback and input.
 - Document the assumptions, limitations, and uncertainties of the risk evaluation process and results, and update them as the project progresses.
 - Review and revise the risk evaluation results periodically, and report any changes or deviations to the stakeholders.

Strategic Program Management in Software Project Management

- Strategic program management is a **strategic management approach** to executing and controlling multiple related projects that share a common goal or vision .
- The goal of strategic program management is to **drive benefits** to the entire program by sharing project resources, costs and activities . It also helps to **align** the projects with the strategic business goals and objectives of the organization.
- A strategic program manager is responsible for **coordinating** and **directing** the projects within the program, as well as **communicating** with the stakeholders and project managers . They also have to **view** and **manage** the interdependencies, risks and issues among the projects .
- A strategic program management office (PMO) is an **organizational component** that can support the strategic program management by providing **guidance, standards, tools** and **best practices** for project management. It can also help to **monitor** and **evaluate** the performance and outcomes of the projects and the program.
- Some of the **benefits** of strategic program management are:

- It connects the projects to the strategic business goals and objectives of the organization.
- It views and manages the interdependencies, risks and issues among the projects.
- It simplifies the resource allocation and optimization across the projects.
- It improves the quality and consistency of the project deliverables and outcomes.
- It enhances the stakeholder engagement and satisfaction.
- Some of the **challenges** of strategic program management are:
 - It requires a high level of leadership, communication and collaboration skills.
 - It involves a complex and dynamic environment with multiple stakeholders and variables.
 - It demands a clear and shared vision and scope for the program and the projects.
 - It faces the risk of scope creep, cost overrun and schedule delay.
- A possible **mnemonic** to remember the key aspects of strategic program management is:

Strategic: Aligns the projects with the strategic business goals and objectives.

Program: Coordinates and directs multiple related projects with a common goal or vision.

Management: Drives benefits to the entire program by sharing resources, costs and activities.

Office: Supports the program management by providing guidance, standards, tools and best practices.

Stepwise Project Planning in spm

- Stepwise project planning (SPP) is a method of software project management that divides a project into smaller and manageable steps, each with a clear goal, deliverable, and deadline.
- SPP is based on the principle of progressive elaboration, which means that the project plan is refined and updated as more information becomes available and the project scope is better defined.
- SPP helps to reduce uncertainty, risk, and complexity in software projects by breaking down the work into smaller chunks that can be easily monitored, controlled, and adjusted.
- SPP consists of the following steps:

1. **Define the project objective:** This step involves identifying the main purpose, scope, and expected outcome of the project. The project objective should be specific, measurable, achievable, relevant, and time-bound (SMART).
2. **Identify the project stakeholders:** This step involves identifying the people or groups who have an interest or influence in the project, such as customers, users, sponsors, managers, developers, testers, etc. The project stakeholders should be consulted and involved in the project planning process to ensure their needs and expectations are met.
3. **Define the project deliverables:** This step involves defining the tangible and intangible products or services that the project will produce, such as software, documentation, training, support, etc. The project deliverables should be aligned with the project objective and the stakeholder requirements.
4. **Estimate the project resources:** This step involves estimating the amount and type of resources that the project will need, such as human, material, financial, technical, etc. The project resources should be realistic, sufficient, and available for the project execution.
5. **Create the project schedule:** This step involves creating a timeline that shows the sequence and duration of the project activities, tasks, and milestones. The project schedule should be based on the project deliverables, resources, and dependencies. The project schedule should also include contingency plans for dealing with potential risks and changes.
6. **Create the project budget:** This step involves creating a financial plan that shows the estimated cost and revenue of the project. The project budget should be based on the project resources, schedule, and quality standards. The project budget should also include contingency reserves for dealing with potential risks and changes.
7. **Create the project quality plan:** This step involves creating a plan that defines the quality criteria, standards, and measures that the project will follow to ensure the project deliverables meet the stakeholder expectations and requirements. The project quality plan should also include quality assurance and quality control activities, such as reviews, audits, tests, etc.
8. **Create the project communication plan:** This step involves creating a plan that defines the communication methods, channels, frequency, and content that the project will use to share information and feedback with the project stakeholders. The project communication plan should also include communication roles and responsibilities, escalation procedures, and reporting formats.
9. **Create the project risk plan:** This step involves creating a plan that identifies, analyzes, prioritizes, and mitigates the potential risks that may affect the project objectives, deliverables, resources, schedule, budget, quality, or communication. The project risk plan should

also include risk response strategies, risk monitoring and control activities, and risk registers.

10. **Create the project change plan:** This step involves creating a plan that defines the change management process that the project will follow to handle any changes that may occur in the project scope, deliverables, resources, schedule, budget, quality, or communication. The project change plan should also include change request procedures, change approval authorities, change impact analysis, and change logs.
- SPP is an iterative and adaptive process that can be repeated and refined throughout the project lifecycle as new information and feedback are obtained and the project requirements and environment change.
 - SPP can be applied to any type of software project, regardless of the size, complexity, or methodology. However, SPP may require more time and effort in the initial planning phase, and may not be suitable for projects that have very tight deadlines, fixed budgets, or rigid specifications.

Unit - 2 - Project Life Cycle and Effort Estimation for the notes of the course Software Project Management

- Project life cycle is the sequence of phases that a project goes through from its initiation to its closure. It provides a framework for managing the project activities and deliverables.
- Project life cycle can vary depending on the type, size, scope, and complexity of the project. However, a common project life cycle model consists of four phases: initiating, planning, executing, and closing.
- Initiating phase is where the project is defined, authorized, and funded. It involves identifying the project objectives, scope, stakeholders, risks, assumptions, and constraints. It also involves selecting the project manager and the project team, and developing the project charter.
- Planning phase is where the project is planned in detail, including the scope, schedule, budget, quality, resources, communication, risk, procurement, and stakeholder management plans. It also involves defining the project deliverables, milestones, tasks, dependencies, and baselines.
- Executing phase is where the project deliverables are produced, monitored, and controlled. It involves executing the project plans, managing the project team, performing quality assurance, communicating with stakeholders, managing risks, and procuring resources.
- Closing phase is where the project is finalized, reviewed, and closed. It involves delivering the project deliverables to the customer, obtaining the customer acceptance, conducting the project evaluation, performing the

lessons learned, releasing the project resources, and archiving the project documents.

- Effort estimation is the process of predicting how much time, energy, or money a project or a task will require. It is used to help draft project plans and budgets, allocate resources, monitor progress, and control costs.
- Effort estimation can be done at different levels of detail and accuracy, depending on the stage of the project and the availability of information. It can be done at the project level, the phase level, the deliverable level, or the task level.
- Effort estimation can be done using various methods and techniques, such as expert judgment, analogy, parametric, bottom-up, top-down, three-point, or Delphi. Each method has its own advantages and disadvantages, and the choice of the method depends on the project characteristics, the estimator's experience, and the stakeholder's expectations.
- Effort estimation can be improved by using historical data, applying estimation standards, involving multiple experts, adjusting for risks and uncertainties, and updating the estimates regularly. It can also be improved by using tools and software that can automate and facilitate the estimation process.

: <https://www.wrike.com/project-management-guide/faq/what-is-effort-estimation/>
<https://consultyellowbrick.com/cost-estimation-through-the-life-cycle-of-a-project/>
<https://www.teamwork.com/blog/project-life-cycle/>
<https://www.coursera.org/articles/project-management-lifecycle>
<https://www.projectmanager.com/blog/level-of-effort>

Software Process and Process Models

- A software process is a set of activities, methods, practices, and transformations that people use to develop and maintain software and its associated products.
- A software process model is an abstract representation of a software process that describes the process from a particular perspective.
- Software process models can be classified into two types: prescriptive and descriptive.
 - Prescriptive models specify the activities, roles, tasks, products, and standards that should be followed in a software process. They are often based on best practices and guidelines from experts and standards bodies. Examples of prescriptive models are waterfall, incremental, iterative, agile, and spiral models.
 - Descriptive models describe how software is actually developed in practice. They are often based on empirical observations and data

from real projects. Examples of descriptive models are personal software process (PSP), team software process (TSP), and capability maturity model integration (CMMI).

- The choice of a software process model depends on various factors, such as the type, size, complexity, and criticality of the software project, the customer requirements and expectations, the available resources and skills, the organizational culture and structure, and the external constraints and risks.
- Some general principles for selecting a software process model are:
 - There is no one-size-fits-all model. Different models may suit different projects and contexts.
 - A software process model should be adaptable and flexible to accommodate changes and uncertainties in the project environment.
 - A software process model should be aligned with the business goals and values of the stakeholders.
 - A software process model should be evaluated and improved continuously based on feedback and lessons learned from the project outcomes and experiences.
- A mnemonic to remember the types of software process models is **PAID**:
 - **P**rescriptive models: prescribe what to do and how to do it.
 - **A**gile models: emphasize flexibility, collaboration, and customer satisfaction.
 - **I**terative models: divide the project into smaller cycles of development and delivery.
 - **D**escriptive models: describe what is actually done and how it is done.

Choice of Process Models in Software Project Management

- A software process model is a representation of the activities, tasks, and resources involved in the development and maintenance of software products.
- A software process model can help to plan, organize, monitor, and control the software project, as well as to improve the quality and productivity of the software development process.
- There are many kinds of software process models, each with its own advantages, disadvantages, and suitability for different types of projects and business applications.
- Some of the popular and important software process models are:
 - **Waterfall model**: A linear and sequential model that follows a fixed set of phases, such as requirements analysis, design, implementation, testing, and deployment. Each phase must be completed and verified before moving to the next phase. This model is simple, easy to understand, and suitable for well-defined and stable projects with clear

and fixed requirements. However, this model does not accommodate changes easily, does not allow feedback from customers or users until the end of the project, and does not deliver working software until the final phase.

- **V model:** An extension of the waterfall model that emphasizes the verification and validation of each phase. For each development phase, there is a corresponding testing phase, such as unit testing, integration testing, system testing, and acceptance testing. This model ensures that the quality of the software is checked at every stage, and that the defects are detected and fixed early. However, this model also suffers from the same limitations as the waterfall model, such as rigidity, inflexibility, and delayed feedback.
- **Incremental model:** A model that divides the software project into smaller and manageable increments, each of which delivers a partial but functional software product. Each increment follows the waterfall or V model, and adds more features and functionality to the software until the final product is complete. This model allows early delivery of working software, frequent feedback from customers or users, and better accommodation of changes. However, this model requires careful planning and coordination of the increments, and may result in inconsistent or incompatible software components if the requirements are not well-defined or stable.
- **RAD model:** A model that stands for Rapid Application Development, and focuses on delivering software products quickly and iteratively, using prototyping, reuse, and automation. This model involves four phases: requirements planning, user design, construction, and cutover. This model is suitable for projects with dynamic and changing requirements, and for customers or users who want to see and use the software as soon as possible. However, this model may compromise the quality and scalability of the software, and may require more resources and skilled developers.
- **Agile model:** A model that follows a set of principles and values that emphasize customer satisfaction, collaboration, flexibility, and responsiveness. This model breaks down the software project into small and iterative cycles, called sprints, each of which delivers a potentially shippable software product. This model involves frequent communication and feedback among the stakeholders, continuous integration and testing, and adaptation to changing requirements and environments. This model is suitable for projects with complex and uncertain requirements, and for customers or users who want to be involved in the software development process. However, this model may require more commitment and discipline from the developers, and may not be compatible with some organizational cultures or processes.
- **Iterative model:** A model that repeats the software development process for each iteration, with the aim of improving the software

product based on the feedback and evaluation from the previous iteration. Each iteration follows the waterfall or V model, and produces a refined and enhanced software product. This model allows incremental improvement of the software quality and functionality, and better management of risks and uncertainties. However, this model may also require more time and resources, and may result in rework and duplication of efforts.

- **Prototype model:** A model that involves creating a simplified and incomplete version of the software product, called a prototype, to demonstrate and validate the requirements and design of the software with the customers or users. The prototype can be either thrown away or evolved into the final software product, depending on the feedback and satisfaction of the customers or users. This model helps to clarify and refine the requirements and design of the software, and to reduce the gap between the expectations and the reality of the software. However, this model may also lead to unrealistic or inaccurate prototypes, and may cause confusion or dissatisfaction among the customers or users.
- The choice of the most appropriate software process model for a project depends on various factors, such as:
 - The nature and scope of the project
 - The requirements and expectations of the customers or users
 - The characteristics and preferences of

Rapid Application Development in Software Project Management

Rapid Application Development (RAD) is a software development methodology that focuses on fast and iterative delivery of working software products. It is based on the following principles :

- Gathering requirements using workshops or focus groups with users and stakeholders
- Prototyping and early, reiterative user testing of designs
- The re-use of software components and frameworks
- A rigidly paced schedule that defers design improvements to the next product version
- Less formality in reviews and other team communication

The RAD methodology consists of four main phases :

- **Planning:** In this phase, the project scope, objectives, budget, and schedule are defined. The project team is also formed and the roles and responsibilities are assigned. The project manager facilitates the communication and collaboration among the team members and the users.
- **User design:** In this phase, the user requirements are elicited and analyzed using workshops or focus groups. The user feedback is used to

create prototypes of the software product, which are then tested and refined with the users. The prototypes serve as the basis for the final design specifications.

- **Construction:** In this phase, the software product is developed using pre-built, reusable code libraries and frameworks. The development process is incremental and iterative, meaning that the product is built in small chunks and tested frequently. The user feedback is incorporated into the development cycle to ensure that the product meets the user expectations and needs.
- **Cutover:** In this phase, the software product is deployed to the production environment and the users are trained on how to use it. The project team also performs the final testing, documentation, and maintenance of the product. Any design improvements or changes are deferred to the next product version.

Some of the advantages of using the RAD methodology are :

- Faster delivery of working software products
- Higher user satisfaction and involvement
- Reduced development costs and risks
- Increased flexibility and adaptability to changing requirements
- Improved quality and functionality of the software products

Some of the disadvantages of using the RAD methodology are :

- Requires highly skilled and experienced developers and users
- Requires frequent and effective communication and collaboration among the project team and the users
- Requires access to reliable and reusable code libraries and frameworks
- May compromise on some features or quality aspects due to time constraints
- May not be suitable for large, complex, or critical software projects

Some of the examples of software products that can be developed using the RAD methodology are:

- Web applications
- Mobile applications
- Business intelligence applications
- Customer relationship management applications
- E-commerce applications

Some of the applications of the RAD methodology in software project management are:

- Developing software products that have a short life cycle or need frequent updates
- Developing software products that have unclear or evolving user requirements

- Developing software products that have a high user involvement and feedback
- Developing software products that have a low to medium complexity and risk

Some of the mnemonics and learning tricks for the RAD methodology are:

- RAD stands for Rapid Application Development, which means developing software products quickly and iteratively
- RAD has four phases: Planning, User design, Construction, and Cutover. A mnemonic to remember them is **PUC-C** (pronounced as “puck-see”)
- RAD is based on five principles: Gathering requirements, Prototyping, Re-using components, Pacing schedule, and Less formality. A mnemonic to remember them is **GPR-PL** (pronounced as “grip-pull”)

Agile Methods in Software Project Management

- Agile methods are a set of software development approaches that are based on the principles of **adaptive planning, iterative delivery, continuous improvement, and customer collaboration**.
- Agile methods aim to deliver software products that meet the changing and evolving needs of the customers, while also ensuring high quality, fast delivery, and customer satisfaction.
- Agile methods are contrasted with traditional or plan-driven methods, which are based on the principles of **predictive planning, sequential delivery, rigid control, and contract negotiation**.
- Traditional methods assume that the requirements, scope, and design of the software product are fixed and well-defined at the beginning of the project, and that the development process follows a linear and sequential order of phases, such as analysis, design, implementation, testing, and deployment.
- Traditional methods are suitable for projects that have stable and clear requirements, low uncertainty, and minimal changes. However, they are often criticized for being too rigid, slow, bureaucratic, and unable to cope with the dynamic and complex nature of software development.
- Agile methods, on the other hand, assume that the requirements, scope, and design of the software product are uncertain and emergent, and that the development process follows an iterative and incremental order of cycles, such as planning, analysis, design, implementation, testing, and feedback.
- Agile methods are suitable for projects that have volatile and ambiguous requirements, high uncertainty, and frequent changes. They are often praised for being flexible, fast, responsive, and customer-oriented.

- Some of the common agile methods are:
 - **Scrum:** A framework that organizes the development process into fixed-length iterations called sprints, each of which delivers a potentially shippable product increment. Scrum defines three roles: the product owner, who represents the customer and defines the product vision and backlog; the scrum master, who facilitates the scrum process and removes impediments; and the development team, who self-organizes and delivers the product increment. Scrum also defines four events: the sprint planning, where the team selects and commits to the backlog items for the sprint; the daily scrum, where the team coordinates and synchronizes their work; the sprint review, where the team demonstrates and inspects the product increment; and the sprint retrospective, where the team reflects and improves their process.
 - **Extreme Programming (XP):** A methodology that focuses on delivering high-quality software through engineering practices, such as test-driven development, pair programming, continuous integration, collective code ownership, refactoring, and simple design. XP also emphasizes customer involvement, frequent releases, user stories, planning game, and feedback loops.
 - **Kanban:** A method that visualizes the workflow of the development process using a board with columns representing different stages, such as to-do, in progress, and done. Kanban limits the amount of work in progress (WIP) in each stage, and encourages the team to pull work from the previous stage when there is capacity. Kanban aims to optimize the flow of work, reduce waste, and improve efficiency and quality.
 - **Lean Software Development:** A philosophy that applies the principles of lean manufacturing to software development, such as eliminate waste, amplify learning, decide as late as possible, deliver as fast as possible, empower the team, build integrity in, and optimize the whole. Lean software development advocates the use of value stream mapping, pull systems, kanban, kaizen, and set-based design to deliver software products that maximize customer value and minimize waste.
 - **Feature-Driven Development (FDD):** A model-driven methodology that organizes the development process into five activities: develop an overall model, build a feature list, plan by feature, design by feature, and build by feature. FDD focuses on delivering software products that consist of a set of features, which are small, client-valued functions. FDD relies on domain experts, chief programmers, class owners, and feature teams to model, design, and implement the features.
 - **Dynamic Systems Development Method (DSDM):** A framework that combines the best practices of agile and traditional meth-

ods, such as timeboxing, prioritization, prototyping, testing, configuration management, and quality management. DSDM defines four phases: feasibility, business study, functional model iteration, and design and build iteration. DSDM also defines nine principles: active user involvement, empowered teams, frequent delivery, integrated testing, stakeholder collaboration, iterative and incremental development, requirements baseline, quality focus, and business value.

- Some of the benefits of agile methods are:
 - They can deliver software products that meet the changing and evolving needs of the customers, by incorporating feedback and learning from each iteration.
 - They can improve the quality of the software products, by applying engineering practices,

Dynamic System Development Method in spm

- Dynamic System Development Method (DSDM) is an agile project delivery framework, initially used as a software development method.
- It provides a structure for building and maintaining systems that meet the needs of stakeholders in a timely and cost-effective manner .
- It is based on the following principles :
 - Focus on the business need: The project should deliver what the business requires, not what the technical team prefers.
 - Deliver on time: The project should adhere to the agreed schedule and budget, using timeboxing and prioritization techniques.
 - Collaborate: The project should involve active participation and feedback from all stakeholders, including users, developers, managers, and testers.
 - Never compromise quality: The project should ensure that the delivered products meet the predefined quality standards and acceptance criteria.
 - Build incrementally from firm foundations: The project should start with a clear and stable vision and scope, and then develop the system in small and frequent increments.
 - Develop iteratively: The project should use feedback loops and testing to refine and improve the system throughout the development process.
 - Communicate continuously and clearly: The project should use various communication channels and methods to share information and knowledge among the project team and stakeholders.
 - Demonstrate control: The project should use effective planning, monitoring, and reporting mechanisms to ensure that the project is under control and aligned with the business objectives.
- It consists of the following phases :
 - Pre-project: This phase involves identifying the project sponsor, the

- project vision, and the feasibility of the project.
- Project life cycle: This phase involves the following stages:
 - * Feasibility study: This stage involves analyzing the business problem, the potential solutions, and the viability of the project.
 - * Business study: This stage involves defining the business requirements, the system architecture, and the project plan.
 - * Functional model iteration: This stage involves creating and testing a functional prototype of the system, based on the prioritized requirements.
 - * Design and build iteration: This stage involves designing and building a complete and integrated system, based on the functional prototype and the quality standards.
 - * Implementation: This stage involves deploying the system to the live environment, training the users, and providing support.
 - Post-project: This phase involves reviewing the project outcomes, identifying the lessons learned, and ensuring the benefits realization.
 - It uses the following roles :
 - Executive sponsor: This role is responsible for providing the strategic direction, the funding, and the high-level approval for the project.
 - Visionary: This role is responsible for defining the vision and the scope of the project, and ensuring that the system meets the business needs.
 - Ambassador user: This role is responsible for representing the interests and expectations of the end users, and providing feedback and validation for the system.
 - Advisor user: This role is responsible for providing domain knowledge and advice to the project team, and supporting the ambassador user.
 - Project manager: This role is responsible for planning, organizing, and controlling the project activities, and ensuring that the project delivers on time, on budget, and on quality.
 - Technical coordinator: This role is responsible for coordinating the technical aspects of the project, such as the system architecture, the design, and the integration.
 - Solution developer: This role is responsible for developing and testing the system components, and ensuring that they meet the functional and non-functional requirements.
 - Solution tester: This role is responsible for testing the system components and the integrated system, and ensuring that they meet the quality standards and the acceptance criteria.
 - Business analyst: This role is responsible for eliciting, analyzing, and documenting the business requirements, and ensuring that they are aligned with the business objectives and the system vision.
 - Workshop facilitator: This role is responsible for facilitating the workshops and meetings among the project team and stakeholders, and ensuring that they are productive and collaborative.
 - DSDM coach: This role is responsible for providing guidance and

support to the project team and stakeholders on the application of the DSDM framework and principles.

- It uses the following techniques :
 - MoSCoW prioritization: This technique involves categorizing the requirements into four levels of importance: Must have, Should have, Could have, and Won't have.
 - Timeboxing: This technique involves dividing the project into fixed-length periods of time, each with a specific goal and deliverable.
 - Prototyping

Extreme Programming in spm

- Extreme Programming (XP) is an agile software development framework that aims to produce higher quality software, and higher quality of life for the development team.
- XP is based on five values: communication, simplicity, feedback, courage, and respect .
- XP follows five rules: planning, managing, designing, coding, and testing.
- XP implements 12 practices: the planning game, small releases, metaphor, simple design, testing, refactoring, pair programming, collective ownership, continuous integration, 40-hour week, on-site customer, and coding standards .
- XP is suitable for projects that have changing or unclear requirements, small and medium-sized teams, and customers who are willing to be involved in the development process .
- XP has some advantages, such as: faster delivery, higher customer satisfaction, lower defect rates, improved teamwork, and increased flexibility .
- XP also has some disadvantages, such as: higher risk of scope creep, lower documentation, higher dependency on individual skills, and potential resistance to change .
- A possible mnemonic to remember the 12 XP practices is: **Plan Small Metaphors Simply Test Refactor Pair Collect Continuously Work Customer Code**. The first letters of each word form the acronym PSMSTRPCCWCC, which can be pronounced as “Piss Mister Pickwick”.
- An example of an application of XP is the development of the Chrysler Comprehensive Compensation System (C3), which was a payroll system for 10,000 employees. The project used XP practices such as on-site customer, pair programming, testing, and refactoring to deliver a working system in four months.

Managing Interactive Processes in spm

- Interactive processes are those that require user input or interaction during their execution, such as graphical user interfaces, web applications, games, etc.

- Managing interactive processes in spm involves designing, developing, testing, deploying, and maintaining such processes to meet the user requirements and expectations.
- Some of the challenges of managing interactive processes in spm are:
 - Balancing the trade-offs between functionality, usability, performance, security, and reliability.
 - Handling the complexity and diversity of user needs, preferences, and behaviors.
 - Adapting to the changing user and environmental contexts, such as device types, network conditions, location, etc.
 - Ensuring the compatibility and interoperability of the interactive processes with other systems and platforms.
 - Evaluating the user satisfaction and feedback, and incorporating them into the improvement cycle.
- Some of the techniques and tools for managing interactive processes in spm are:
 - User-centered design (UCD), which is an approach that involves the user throughout the development process, from identifying the user needs and goals, to prototyping and testing the interactive processes, to evaluating the user experience and satisfaction.
 - Agile methods, which are iterative and incremental approaches that allow for frequent delivery and feedback, and embrace change and adaptation.
 - Model-driven development (MDD), which is an approach that uses models and abstractions to specify, generate, and verify the interactive processes, and reduce the gap between the design and the implementation.
 - Software architectures, which are the high-level structures and patterns that define the components, interactions, and properties of the interactive processes, and provide the basis for their quality attributes and evolution.
 - Software testing, which is the process of verifying and validating the functionality, usability, performance, security, and reliability of the interactive processes, and identifying and correcting the defects and errors.
 - Software deployment, which is the process of delivering and installing the interactive processes to the target platforms and environments, and ensuring their availability and accessibility.
 - Software maintenance, which is the process of modifying and updating the interactive processes to fix the problems, improve the quality, and adapt to the changes.

Basics of Software Estimation in spm

- Software estimation is the process of predicting the most realistic amount of effort, cost, and time required to complete a software project.

- Software estimation is an essential part of software project management (SPM), which involves planning, organizing, monitoring, and controlling the software development process.
- Software estimation helps to set realistic expectations for the stakeholders, allocate resources, schedule tasks, track progress, and evaluate quality of the software product.
- Software estimation is not an exact science, but rather a skill that requires experience, judgment, and knowledge of various estimation techniques and tools.
- Some of the common challenges in software estimation are:
 - Uncertainty and complexity of software requirements
 - Lack of historical data and benchmarks
 - Human factors such as motivation, productivity, and skill level
 - Environmental factors such as technology, tools, and processes
 - Risks and contingencies that may affect the project outcome
- Some of the common software estimation techniques are:
 - Expert judgment: Based on the opinion and experience of one or more experts who have done similar projects before.
 - Algorithmic models: Based on mathematical formulas that relate the software size, effort, and cost to various project attributes and parameters. Examples are COCOMO, Function Point Analysis, and Use Case Points.
 - Estimation by analogy: Based on comparing the new project to one or more similar projects that have been completed in the past and adjusting the effort and cost accordingly.
 - Top-down and bottom-up: Based on decomposing the project into smaller components and estimating them separately, either from the highest level to the lowest level (top-down) or vice versa (bottom-up).
 - Parametric models: Based on using statistical methods and data analysis to derive the relationship between the software size, effort, and cost and various project factors. Examples are SLIM, SEER-SEM, and PRICE-S.
- Some of the common software estimation tools are:
 - ANGEL: A tool that uses case-based analogy to estimate software effort based on the Euclidean distance between the new project and the source projects.
 - COCOMO II: A tool that implements the COCOMO model to estimate software effort, cost, and schedule based on the software size, complexity, and quality factors.
 - Function Point WORKBENCH: A tool that uses function point analysis to measure the software size and complexity based on the user requirements and the data and transaction functions.
 - Use Case Points Calculator: A tool that uses use case points to estimate software effort and cost based on the number and complexity of use cases and actors in the software system.
 - SLIM-Estimate: A tool that uses the SLIM model to estimate soft-

ware effort, cost, and schedule based on the software size, productivity, and quality factors.

Effort and Cost Estimation Techniques in spm

- Software Project Management (SPM) is the process of planning, organizing, monitoring, and controlling software projects.
- One of the most important part in SPM is cost and effort estimation. Cost estimation is the process of predicting the amount of money required to develop a software project. Effort estimation is the process of predicting the amount of human resources and time required to complete a software project.
- Accurate cost and effort estimation is critical for both developers and customers to make informed decisions, allocate resources, set budgets, and schedule deadlines. However, conducting cost and effort estimation is not an easy task, as software projects are often complex, uncertain, and dynamic.
- There are various techniques and methods for cost and effort estimation, which can be classified into three categories: algorithmic, expert judgment, and analogy.

Algorithmic Techniques

- Algorithmic techniques are based on mathematical models that use historical data and project parameters as inputs to calculate the cost and effort of a software project. The most widely used algorithmic technique is the COCOMO (Constructive Cost Model) model, which was developed by Barry Boehm in 1981 and revised in 2000.
- The COCOMO model consists of three levels: basic, intermediate, and detailed. The basic level uses only the estimated size of the software project (in lines of code or function points) as the input. The intermediate level uses the size and a set of 15 cost drivers (such as product complexity, personnel capability, development environment, etc.) as the inputs. The detailed level uses the size, the cost drivers, and a set of 17 effort multipliers (such as required reliability, database size, documentation, etc.) as the inputs.
- The COCOMO model calculates the effort (in person-months) and the duration (in months) of a software project using the following equations:
 - $\text{Effort} = a * (\text{Size})^b$
 - $\text{Duration} = c * (\text{Effort})^d$
 - Where a, b, c, and d are constants that depend on the level and the mode of the project (organic, semi-detached, or embedded).

- The COCOMO model has some advantages and disadvantages. Some of the advantages are:
 - It is based on empirical data and validated by many studies.
 - It is easy to use and understand.
 - It can be calibrated to fit different organizations and environments.
 - It can handle different types of software projects and development methods.
- Some of the disadvantages are:
 - It relies on the accuracy of the size estimation, which is often difficult and subjective.
 - It assumes a linear relationship between size and effort, which may not hold for very large or very small projects.
 - It does not account for some factors that may affect the cost and effort, such as risk, quality, change, and innovation.

Expert Judgment Techniques

- Expert judgment techniques are based on the opinions and experiences of experts who have knowledge and expertise in software development and estimation. The experts can be internal (such as project managers, developers, analysts, etc.) or external (such as consultants, customers, users, etc.).
- The expert judgment techniques can be performed in various ways, such as:
 - Delphi method: A structured process that involves multiple rounds of anonymous questionnaires and feedback among a group of experts, until a consensus is reached.
 - Wideband Delphi method: A variation of the Delphi method that involves face-to-face meetings and discussions among the experts, along with the questionnaires and feedback.
 - Planning poker: A collaborative and interactive process that involves a group of experts who use a deck of cards with numbers to estimate the size, cost, and effort of a software project.
 - Work breakdown structure (WBS): A hierarchical decomposition of a software project into smaller and manageable tasks, which are then estimated by the experts individually or collectively.
- The expert judgment techniques have some advantages and disadvantages. Some of the advantages are:
 - They can capture the tacit knowledge and intuition of the experts, which may not be available in historical data or mathematical models.
 - They can handle complex, uncertain, and novel software projects, which may not fit the assumptions or parameters of algorithmic tech-

niques.

- They can incorporate qualitative and subjective factors, such as risk, quality, change, and innovation, which may affect the cost and effort of a software project.
- Some of the disadvantages are:
 - They depend on the availability, reliability, and consistency of the experts, which may vary across different projects and contexts.
 -

COSMIC Full Function Points in spm

- COSMIC stands for Common Software Measurement International Consortium, which is an organization that develops and maintains a standard method for measuring software functional size.
- COSMIC function points are a unit of measure of software functional size, which is the amount of functionality that a software provides to its users, based on their requirements .
- The functional size is consistent regardless of the technology used to build the software, and can be estimated or measured at any stage of the software life cycle .
- The functional size measurement (FSM) process using COSMIC method involves identifying and counting four types of data movements between the software and its functional users :
 - Entry: an input of data from a functional user to the software
 - Exit: an output of data from the software to a functional user
 - Read: a retrieval of data from persistent storage by the software
 - Write: a storage of data to persistent storage by the software
- Each data movement is counted as one COSMIC function point (CFP), regardless of the complexity or volume of the data .
- The software functional users can be human users, other software systems, or hardware devices that interact with the software .
- The software is divided into functional processes, which are groups of data movements that contribute to a user-recognizable function .
- The functional size of the software is the sum of the CFPs of all the functional processes .
- The COSMIC method can be applied to a wide range of software domains, such as business, real-time, embedded, and infrastructure software .
- The COSMIC method can be used for various purposes, such as estimating effort, cost, and duration of software projects, benchmarking productivity and quality, and evaluating software maintenance .

Example of COSMIC FSM

Consider a simple software that allows a user to create, read, update, and delete (CRUD) records of employees in a database. The software has the following

functional processes and data movements:

- Create employee: the user enters the employee data (Entry), the software validates the data (Exit), and stores the data in the database (Write).
- Read employee: the user enters the employee ID (Entry), the software retrieves the employee data from the database (Read), and displays the data to the user (Exit).
- Update employee: the user enters the employee ID (Entry), the software retrieves the employee data from the database (Read), displays the data to the user (Exit), the user modifies the data (Entry), the software validates the data (Exit), and updates the data in the database (Write).
- Delete employee: the user enters the employee ID (Entry), the software retrieves the employee data from the database (Read), displays the data to the user (Exit), the user confirms the deletion (Entry), and the software deletes the data from the database (Write).

The functional size of the software is calculated as follows:

Functional process	Entry	Exit	Read	Write	CFP
Create employee	1	1	0	1	3
Read employee	1	1	1	0	3
Update employee	2	2	1	1	6
Delete employee	2	1	1	1	5
Total	6	5	3	3	17

Therefore, the functional size of the software is 17 CFPs.

Advantages of COSMIC method

- It is simple, intuitive, and easy to learn and apply

COCOMO II in spm

- COCOMO II stands for **COnstructive COst MOdel II** and is a model that allows one to estimate the cost, effort, and schedule when planning a new software development activity.
- COCOMO II is the revised version of the original COCOMO (COCOMO 81) model published in 1981. It is developed at the University of Southern California.
- COCOMO II is designed to address software development practices in the 1990s and 2000s, such as rapid application development, reuse, re-engineering, and object-oriented development.
- COCOMO II consists of three sub-models:
 - **Application Composition Model:** This model is used for projects that are based on rapid application development using graphical user interface builders, code generators, or other composition tools. It

estimates the effort based on the number of object points, which are a measure of the functionality and complexity of the software components.

- **Early Design Model:** This model is used for projects that are in the early stages of design, where the requirements are not fully defined and the architecture is not yet decided. It estimates the effort based on the size of the software in terms of unadjusted function points, which are a measure of the user functions and data processed by the software. It also considers the product, platform, personnel, and project attributes that affect the productivity and quality of the software.
- **Post-Architecture Model:** This model is used for projects that have a defined architecture and a detailed design. It estimates the effort based on the size of the software in terms of source lines of code, which are a measure of the physical volume of the software. It also considers the cost drivers that affect the effort, such as the required reliability, complexity, reuse, documentation, and team experience.
- COCOMO II uses the following formula to estimate the effort in person-months (PM):
 - $PM = A * Size^E * M$
 - Where:
 - * A is a constant that depends on the sub-model used
 - * Size is the size of the software in object points, function points, or source lines of code
 - * E is an exponent that depends on the sub-model used and the scale factors that reflect the project characteristics
 - * M is a multiplier that depends on the cost drivers that affect the effort
- COCOMO II uses the following formula to estimate the schedule in months (TDEV):
 - $TDEV = 3 * (PM)^F$
 - Where:
 - * PM is the effort in person-months
 - * F is an exponent that depends on the sub-model used and the scale factors that reflect the project characteristics
- COCOMO II also provides a range of estimates for the effort and schedule based on different levels of confidence, such as most likely, optimistic, and pessimistic.
- COCOMO II is a simple and well-tested model that provides about 20% cost and 70% time estimate accuracy. It can be calibrated and tailored to specific organizations and projects using historical data and expert judgment.
- COCOMO II is useful for estimating the cost, effort, and schedule of software projects of different types, sizes, and complexities. It can also be used for comparing alternative design and development strategies, and for planning and managing software projects.

A Parametric Productivity Model in spm

- A parametric productivity model is a mathematical model that relates the output of a software project to the input factors, such as size, complexity, quality, resources, etc.
- A parametric productivity model can be used to estimate the cost, effort, schedule, and quality of a software project, as well as to compare different alternatives and optimize the project plan.
- A parametric productivity model is based on empirical data and statistical analysis, and it can be calibrated to fit the specific characteristics of a software organization or domain.
- One of the most widely used parametric productivity models in software engineering is COCOMO (Constructive Cost Model), which was developed by Barry Boehm in 1981 and revised in 2000.
- COCOMO consists of three levels of models: basic, intermediate, and detailed, which differ in the number and complexity of the input factors and the equations used to calculate the output.
- The basic COCOMO model estimates the effort (in person-months) required to develop a software project based on the size (in thousands of source lines of code) and the mode (organic, semi-detached, or embedded) of the project. The mode reflects the degree of novelty, flexibility, and reliability required for the project.
- The basic COCOMO model has the following equation:

$$\text{Effort} = a * (\text{Size})^b$$

where a and b are constants that depend on the mode of the project.

- The intermediate COCOMO model refines the basic model by introducing 15 cost drivers that account for the attributes of the product, the hardware, the personnel, and the project. The cost drivers are rated on a scale from very low to very high, and each rating has a corresponding multiplier that affects the effort estimation. The intermediate COCOMO model has the following equation:

$$\text{Effort} = a * (\text{Size})^b * \text{EAF}$$

where EAF is the effort adjustment factor, which is the product of the multipliers of the 15 cost drivers.

- The detailed COCOMO model further refines the intermediate model by dividing the software project into different phases, such as planning, design, coding, testing, and integration, and estimating the effort and schedule for each phase separately. The detailed COCOMO model also considers the reuse and reengineering of existing software components, and the impact of risk analysis and management on the project.

- A mnemonic to remember the three levels of COCOMO models is:
Basic COCOMO: Size and Mode
Intermediate COCOMO: Size, Mode, and Cost Drivers
Detailed COCOMO: Size, Mode, Cost Drivers, and Phases
- A parametric productivity model in spm can be implemented using various software tools, such as SPM, FSL, and AFNI, which allow the user to specify the input factors, perform the calculations, and visualize the results.
- A parametric productivity model in spm can be validated and evaluated using various criteria, such as accuracy, reliability, consistency, simplicity, generality, and adaptability.

Unit - 3 - Activity Planning and Risk Management for the notes of the course Software Project Management

- Activity planning is the process of defining and sequencing the tasks, milestones, and deliverables of a software project.
- Activity planning aims to create a realistic and feasible schedule that meets the project objectives and stakeholder expectations.
- Activity planning involves the following steps:
 - Identify the project scope and objectives
 - Decompose the project into manageable activities or work packages
 - Estimate the effort, duration, and resources required for each activity
 - Identify the dependencies and constraints among the activities
 - Create a network diagram or a Gantt chart to visualize the activity sequence and critical path
 - Review and refine the activity plan with the project team and stakeholders
 - Monitor and control the activity progress and performance
- Activity planning is an iterative and dynamic process that requires constant updates and adjustments based on the project changes and feedback.
- Risk management is the process of identifying, analyzing, and responding to the uncertainties and threats that may affect the software project.
- Risk management aims to minimize the negative impacts and maximize the positive opportunities of the risks on the project objectives and outcomes.
- Risk management involves the following steps:
 - Identify the potential risks that may affect the project scope, schedule, quality, cost, or stakeholder satisfaction

- Analyze the probability and impact of each risk and prioritize them based on their severity and exposure
- Plan the appropriate strategies and actions to avoid, reduce, transfer, or accept each risk
- Implement the risk response plan and assign roles and responsibilities for risk management
- Monitor and review the risk status and effectiveness of the risk response plan
- Communicate and document the risk information and lessons learned
- Risk management is a proactive and continuous process that requires collaboration and communication among the project team and stakeholders.
- A mnemonic to remember the steps of activity planning is **IDECRM** (Identify, Decompose, Estimate, Create, Review, Monitor).
- A mnemonic to remember the steps of risk management is **IAPIMC** (Identify, Analyze, Plan, Implement, Monitor, Communicate).

Objectives of Activity Planning in SPM

- Activity planning is the process of identifying, sequencing, estimating, and scheduling the activities or tasks required to complete a project.
- The objectives of activity planning in SPM are:
 - To define the scope and deliverables of the project in terms of activities or tasks.
 - To determine the logical order and dependencies of the activities or tasks.
 - To estimate the duration, effort, cost, and resources required for each activity or task.
 - To optimize the use of resources and minimize the risk of delays or conflicts.
 - To create a realistic and feasible schedule for the project that meets the stakeholder expectations and constraints.
- A possible mnemonic to remember the objectives of activity planning in SPM is:
 - **D**efine
 - **D**etermine
 - **E**stimate
 - **O**ptimize
 - **C**reate
 - **DDEOC**: **D**o **D**iligent **E**ffort to **O**rganize and **C**oordinate

Project Schedules in spm

- Project schedules are the core of the project plan. They show how the work will be performed, who will do it, when it will be done, and what resources are needed.
- Project schedules are created and tracked with project scheduling software, which has features that allow project managers to monitor the progress of tasks, resources and costs in real time.
- Project schedules are based on the project scope, the work breakdown structure (WBS), the activity list, the activity attributes, the resource requirements, the resource calendars, the activity duration estimates, and the project management plan.
- Project schedules use different techniques to represent the sequence and dependencies of activities, such as network diagrams, Gantt charts, milestone charts, and critical path method (CPM).
- Project schedules are subject to change due to various factors, such as risks, issues, changes, delays, and uncertainties. Project managers need to update and communicate the project schedule regularly to stakeholders and team members.
- Project schedules are part of the schedule management plan (SMP), which is a document that details how the project schedule will be created, managed, and monitored. The SMP includes information about the scheduling methodology, tools, processes, roles, responsibilities, and performance measures.

Some mnemonics and learning tricks for project schedules in spm are:

- To remember the inputs of project schedule development, use the acronym **SWARADP** (Scope, WBS, Activity list, Resource requirements, Activity attributes, Duration estimates, Project management plan).
- To remember the tools and techniques of project schedule development, use the acronym **NAPES** (Network diagrams, Activity resource estimating, Parametric estimating, Expert judgment, Scheduling software).
- To remember the outputs of project schedule development, use the acronym **SAC** (Schedule baseline, Activity attributes updates, Change requests).
- To remember the four types of dependencies between activities, use the acronym **FSLs** (Finish-to-start, Start-to-start, Finish-to-finish, Start-to-finish).
- To remember the six steps of creating a schedule management plan, use the acronym **DAMPER** (Define, Analyze, Monitor, Perform, Evaluate, Report).

Activities in spm

SPM stands for Software Project Management, which is the discipline of planning, organizing, monitoring, and controlling software projects. SPM covers various activities that aim to deliver a software product that meets the requirements of the customer, within the budget and schedule constraints, and with the desired quality and functionality. Some of the main activities in SPM are:

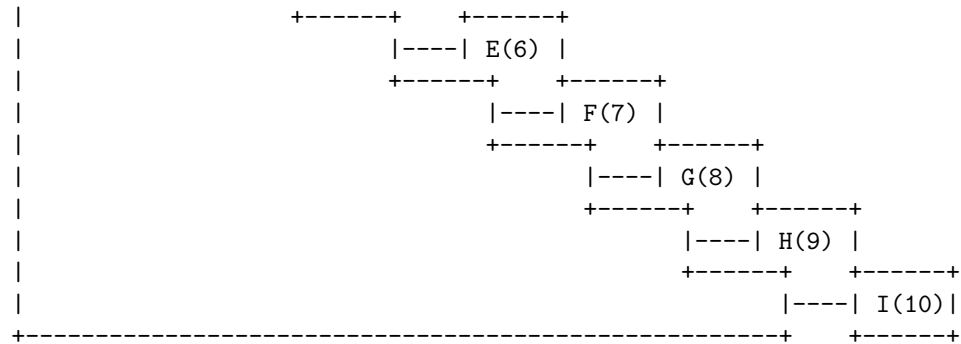
- **Project planning and tracking:** This activity involves defining the scope, objectives, deliverables, milestones, tasks, and resources of the project, as well as estimating the cost, time, and effort required to complete them. Project planning also includes developing a project plan that outlines the work breakdown structure, schedule, budget, risk analysis, quality plan, communication plan, and other relevant documents. Project tracking involves monitoring the progress, performance, and status of the project, as well as identifying and resolving any issues, changes, or deviations that may arise during the project execution.
- **Project resource management:** This activity involves acquiring, allocating, and managing the human, physical, and financial resources that are needed for the project. Resource management also includes optimizing the utilization, productivity, and motivation of the project team, as well as managing the relationships and expectations of the stakeholders, such as the customer, the sponsor, the senior management, and the end users.
- **Scope management:** This activity involves defining, validating, and controlling the features and functions that the software product should provide, as well as the boundaries and constraints that the project should adhere to. Scope management also includes managing the changes and variations that may affect the scope during the project lifecycle, and ensuring that they are aligned with the project objectives and customer needs.
- **Estimation management:** This activity involves estimating the size, complexity, and quality of the software product, as well as the cost, time, and effort required to develop, test, and deploy it. Estimation management also includes applying various techniques, methods, and tools to produce accurate and reliable estimates, as well as adjusting and updating them as the project progresses and more information becomes available.
- **Project risk management:** This activity involves identifying, analyzing, and prioritizing the potential risks that may affect the project, such as technical, operational, organizational, financial, or environmental risks. Risk management also includes developing and implementing strategies and actions to mitigate, avoid, transfer, or accept the risks, as well as monitoring and controlling them throughout the project lifecycle.
- **Scheduling management:** This activity involves defining, sequencing, and assigning the tasks and activities that need to be performed to com-

plete the project, as well as determining the dependencies, durations, and resources for each task. Scheduling management also includes developing and maintaining a project schedule that shows the start and finish dates, milestones, and critical path of the project, as well as tracking and controlling the actual progress and performance of the project against the planned schedule.

- **Project communication management:** This activity involves planning, executing, and controlling the communication processes and channels that are used to exchange information, ideas, feedback, and reports among the project team and the stakeholders. Communication management also includes ensuring that the communication is timely, clear, consistent, and effective, as well as managing the conflicts, issues, and expectations that may arise from the communication.
- **Configuration management:** This activity involves identifying, organizing, storing, and controlling the versions and changes of the software product and the project documents, such as the requirements, design, code, test cases, and user manuals. Configuration management also includes establishing and enforcing the standards, procedures, and tools for managing the configuration items, as well as auditing and reviewing them to ensure their quality and integrity.

Sequencing and Scheduling in spm

- Sequencing and scheduling are two important techniques for planning and managing complex projects.
- Sequencing is the process of determining the order of tasks or activities that need to be performed in a project. It involves identifying the dependencies, constraints, and priorities among the tasks.
- Scheduling is the process of allocating time and resources to the tasks or activities in a project. It involves estimating the duration, start and finish dates, and resource requirements of each task.
- Sequencing and scheduling can be done using various methods and tools, such as network diagrams, Gantt charts, critical path analysis, resource leveling, etc.
- Sequencing and scheduling can help to achieve the following benefits for a project:
 - Optimize the use of resources and reduce costs
 - Minimize the risks and uncertainties
 - Enhance the quality and performance
 - Improve the communication and coordination
 - Monitor and control the progress and status
- Sequencing and scheduling can also pose some challenges for a project, such as:
 - Dealing with changes and uncertainties
 - Balancing the trade-offs and constraints

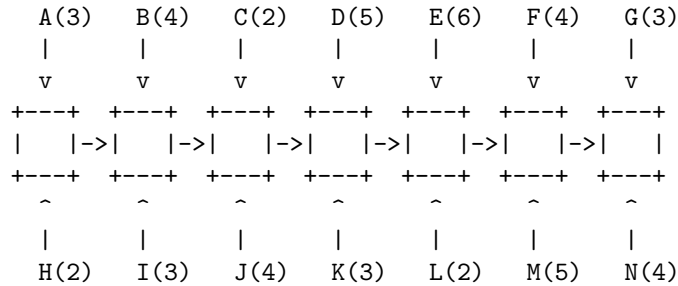


- The numbers in parentheses indicate the duration of each activity in days.
- The critical path is A-B-C-D-E-F-G-H-I, which has a total duration of 54 days.
- A mnemonic to remember the steps of network planning is: **DAD** (Draw, Analyze, Develop).
 - Draw the network diagram using AON or AOA notation.
 - Analyze the network to find the critical path and the slack time for each activity.
 - Develop the project schedule based on the network analysis and the available resources.

Formulating Network Model in spm

- A network model is a graphical representation of the activities and their interrelationships in a software project.
- A network model can help to plan, schedule, monitor and control the project activities, resources and costs.
- A network model can also help to identify the critical path, the slack time, the float time and the project duration.
- There are two main methods of formulating a network model: activity-on-node (AON) and activity-on-arrow (AOA).
- In AON, the activities are represented as nodes (boxes) and the dependencies are represented as arrows (lines) between the nodes.
- In AOA, the activities are represented as arrows (lines) and the events are represented as nodes (circles).
- AON is more commonly used than AOA because it is easier to draw and modify, and it can handle complex dependencies and dummy activities.
- A network model can be constructed by following these steps:
 - Identify the activities and their dependencies. Use a work breakdown structure (WBS) or a precedence diagram to help with this step.
 - Assign a unique identifier and an estimated duration to each activity.
 - Draw the network diagram using either AON or AOA method. Start from the initial node and follow the dependencies until the final node. Make sure there are no loops or isolated nodes in the diagram.

- Label the nodes and the arrows with the activity identifiers and durations.
- Check the network diagram for accuracy and completeness.
- Here is an example of a network model for a software project using AON method:

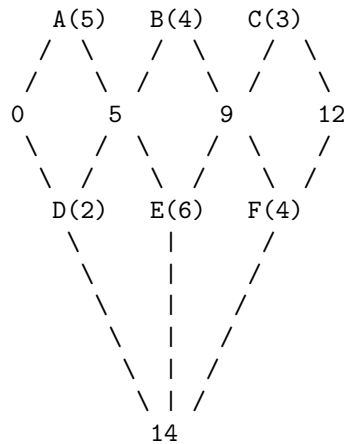


- A mnemonic to remember the steps of formulating a network model is: **IADLC** (Identify, Assign, Draw, Label, Check).

Forward Pass & Backward Pass Techniques in spm

- Forward pass and backward pass are techniques used to calculate the earliest and latest start and finish times of the activities in a project network diagram.
- Forward pass is the process of moving from the first activity to the last activity in the network diagram and calculating the earliest start (ES) and earliest finish (EF) times of each activity.
- Backward pass is the process of moving from the last activity to the first activity in the network diagram and calculating the latest start (LS) and latest finish (LF) times of each activity.
- The difference between the latest and earliest times of an activity is called the total float (TF), which represents the amount of time an activity can be delayed without affecting the project completion time.
- The critical path is the sequence of activities that has the least amount of total float and determines the minimum duration of the project.
- The forward pass and backward pass techniques can be applied to any type of network diagram, such as activity-on-node (AON) or activity-on-arrow (AOA).
- The steps for performing the forward pass and backward pass techniques are as follows:
 - Forward pass:
 - * Start with the first activity in the network diagram and assign it an ES of zero and an EF equal to its duration.

- * For each subsequent activity, calculate its ES as the maximum EF of its predecessors and its EF as the sum of its ES and duration.
- * Repeat this process until you reach the last activity in the network diagram and record its EF as the project completion time.
- Backward pass:
 - * Start with the last activity in the network diagram and assign it an LF equal to its EF and an LS equal to its LF minus its duration.
 - * For each preceding activity, calculate its LF as the minimum LS of its successors and its LS as the difference between its LF and duration.
 - * Repeat this process until you reach the first activity in the network diagram and record its LS as zero.
- Total float and critical path:
 - * For each activity, calculate its TF as the difference between its LF and EF or between its LS and ES.
 - * Identify the activities that have a TF of zero and mark them as critical. These activities form the critical path of the project.
 - * The project completion time is equal to the EF or LF of the last activity on the critical path.
- An example of applying the forward pass and backward pass techniques to an AON network diagram is shown below:



- The ES, EF, LS, LF, and TF values of each activity are shown in the table below:

Activity	ES	EF	LS	LF	TF
A	0	5	0	5	0
B	5	9	5	9	0
C	9	12	9	12	0

Activity	ES	EF	LS	LF	TF
D	0	2	3	5	3
E	9	15	9	15	0
F	12	16	12	16	0

- The critical path of the project is A-B-C-E-F and the project completion time is 16 days.

Critical Path (CPM) Method in spm

- The critical path method (CPM) is a technique where you identify tasks that are necessary for project completion and determine scheduling flexibilities .
- A critical path in project management is the longest sequence of activities that must be finished on time in order for the entire project to be complete .
- The critical path method helps you to plan, schedule, monitor, and control your project effectively .
- The critical path method can help you to:
 - Identify the shortest possible duration of the project
 - Identify the activities that are critical and non-critical for the project
 - Identify the slack or float time for each activity
 - Identify the risks and uncertainties associated with the project
 - Optimize the use of resources and avoid bottlenecks
 - Manage changes and updates to the project schedule
- The critical path method involves six steps:
 1. Specify each activity using the work breakdown structure (WBS)
 2. Establish the sequence of activities and their dependencies
 3. Estimate the duration of each activity
 4. Draw the network diagram of the project
 5. Calculate the earliest and latest start and finish times for each activity
 6. Identify the critical path and the critical activities
- The critical path is the longest path in the network diagram, and it determines the minimum time required to complete the project .
- The critical activities are the activities that lie on the critical path, and they have zero slack or float time .
- The slack or float time is the amount of time that an activity can be delayed without affecting the project duration .
- The network diagram is a graphical representation of the project activities and their dependencies, using nodes and arrows .
- The nodes represent the activities, and the arrows represent the dependencies .
- The dependencies can be of four types :
 - Finish-to-start (FS): The successor activity cannot start until the

- predecessor activity is finished
 - Start-to-start (SS): The successor activity cannot start until the predecessor activity has started
 - Finish-to-finish (FF): The successor activity cannot finish until the predecessor activity is finished
 - Start-to-finish (SF): The successor activity cannot finish until the predecessor activity has started
- The network diagram can be drawn using different methods, such as the activity-on-node (AON) or the activity-on-arrow (AOA) method .
- The activity-on-node (AON) method uses nodes to represent activities and arrows to represent dependencies .
- The activity-on-arrow (AOA) method uses arrows to represent activities and nodes to represent dependencies .
- The network diagram can be used to calculate the earliest and latest start and finish times for each activity, using the forward and backward pass methods .
- The forward pass method calculates the earliest start and finish times for each activity, starting from the first activity and moving forward .
- The backward pass method calculates the latest start and finish times for each activity, starting from the last activity and moving backward .
- The earliest start time (ES) is the earliest time that an activity can start, given the dependencies and the durations of the preceding activities .
- The earliest finish time (EF) is the earliest time that an activity can finish, given the dependencies and the durations of the

Risk Identification in spm

- Risk identification is the process of identifying and documenting the potential sources of uncertainty that may affect the project objectives, such as scope, schedule, cost, and quality.
- Risk identification is an iterative and continuous process that should be performed throughout the project life cycle, from initiation to closure.
- Risk identification is essential for effective risk management, as it provides the basis for risk analysis, risk response planning, and risk monitoring and control.
- Risk identification can be performed using various techniques, such as:
 - Brainstorming: A group technique that involves generating a list of potential risks by encouraging creative and divergent thinking among the participants.
 - Interviews: A technique that involves asking key stakeholders, experts, or team members about their views and experiences on the project risks and their causes and effects.
 - Checklists: A technique that involves using a predefined list of common or historical risks that may apply to the project or similar

projects.

- Questionnaires: A technique that involves using a structured or semi-structured survey to collect information from a large number of respondents on the project risks and their characteristics.
 - SWOT analysis: A technique that involves identifying the strengths, weaknesses, opportunities, and threats of the project or the organization.
 - Assumption analysis: A technique that involves identifying and validating the assumptions that are made during the project planning and execution, and assessing their impact on the project risks.
 - Cause and effect diagrams: A technique that involves using a graphical tool to identify the root causes of a problem or a risk, and their relationships and effects.
 - Nominal group technique: A technique that involves ranking the ideas or risks generated by a group of participants, and reaching a consensus on the most important ones.
 - Delphi technique: A technique that involves using a panel of experts to anonymously provide their opinions and feedback on the project risks, and reaching a consensus through multiple rounds of communication.
 - Scenario analysis: A technique that involves creating and analyzing different possible scenarios or situations that may affect the project outcomes, and identifying the risks associated with each scenario.
 - Sensitivity analysis: A technique that involves examining how the variations in the project variables, such as cost, time, or quality, affect the project objectives or performance.
 - Probability and impact matrix: A technique that involves assessing the likelihood and the severity of each identified risk, and prioritizing them based on their combined effect on the project.
 - Risk breakdown structure: A technique that involves decomposing the project risks into hierarchical levels of categories and subcategories, based on their sources, types, or impacts.
- Some of the benefits of risk identification are:
 - It helps to proactively identify and address the potential threats and opportunities that may affect the project success.
 - It helps to improve the project planning and decision making by providing a realistic and comprehensive view of the project uncertainties and their implications.
 - It helps to enhance the project communication and stakeholder engagement by involving them in the risk identification process and sharing the risk information.
 - It helps to reduce the project costs and delays by avoiding or minimizing the negative impacts of the risks, and exploiting or enhancing the positive impacts of the opportunities.
 - It helps to increase the project quality and customer satisfaction by

delivering the project outputs that meet or exceed the expectations and requirements.

- Some of the challenges of risk identification are:
 - It can be time-consuming and resource-intensive, especially for large and complex projects with multiple stakeholders and uncertainties.
 - It can be influenced by the subjective judgments and biases of the participants, such as overconfidence, anchoring, availability, or confirmation bias.
 - It can be affected by the incomplete or inaccurate information, assumptions, or data that are used for identifying the risks.
 - It can be limited by the scope and quality of the risk identification techniques that are used, and their suitability and applicability to the project context and objectives.

Risk Assessment in spm

- Risk assessment is the process of identifying, analyzing, and evaluating the potential hazards and consequences of a project or activity.
- In spm, risk assessment can be applied to various aspects, such as subsea pipelines, single point mooring (SPM) systems, project management, and software development.
- The purpose of risk assessment in spm is to reduce the uncertainty and improve the decision making for the project or activity, by prioritizing the risks and developing appropriate mitigation or avoidance strategies.
- The steps of risk assessment in spm are:
 1. Identify the risks: This involves brainstorming, reviewing historical data, consulting experts, and using tools such as checklists, fault tree analysis, or failure mode and effects analysis (FMEA) to identify the possible sources of risk for the project or activity.
 2. Analyze the risks: This involves estimating the probability and impact of each risk, using qualitative or quantitative methods, such as risk matrices, probability distributions, or Monte Carlo simulations. The probability is the likelihood of the risk occurring, and the impact is the severity of the consequence if the risk occurs.
 3. Evaluate the risks: This involves ranking the risks according to their priority, based on the combination of probability and impact. The priority can be expressed as low, medium, or high, or as a numerical score. The higher the priority, the more attention the risk requires.
 4. Treat the risks: This involves selecting and implementing the best option to deal with each risk, such as avoiding, transferring, mitigating, or accepting the risk. The option should be feasible, cost-effective, and aligned with the project or activity objectives. The option should also be monitored and reviewed for its effectiveness and efficiency.

- Some examples of risk assessment in spm are:
 - Risk assessment of subsea pipeline due to installation and operation of SPM: This involves identifying the potential hazards and consequences of the subsea pipeline, such as third party factors (dropped anchor, dragged anchor, and ship sinking), corrosion, fatigue, or leakage, and analyzing and evaluating them using the DNVGL RP F107 standard and numerical simulations .
 - Risk assessment of SPM system integrity: This involves identifying the potential failures and damages of the SPM system, such as mooring line breakage, buoyancy loss, or hull cracking, and analyzing and evaluating them using a qualitative risk based approach, based on the SPM design, characteristic, assessment, and inspection data.
 - Risk assessment of project management: This involves identifying the potential uncertainties and challenges of the project, such as scope creep, budget overrun, schedule delay, or stakeholder conflict, and analyzing and evaluating them using tools such as risk registers, risk breakdown structures, or SWOT analysis .
 - Risk assessment of software development: This involves identifying the potential defects and errors of the software, such as functional, performance, security, or usability issues, and analyzing and evaluating them using tools such as risk identification checklists, risk exposure charts, or risk mitigation plans.
- Some mnemonics and learning tricks for risk assessment in spm are:
 - The four steps of risk assessment can be remembered as IATE: Identify, Analyze, Evaluate, and Treat.
 - The four options to treat the risks can be remembered as ATMA: Avoid, Transfer, Mitigate, and Accept.
 - The factors that affect the probability and impact of the risks can be remembered as COPS: Complexity, Opportunity, People, and Stakeholders.

Risk Planning in spm

- Risk planning is the process of identifying, analyzing, and responding to the risks that may affect a software project.
- Risk planning is an important part of software project management (spm) because it helps to reduce the uncertainty and increase the likelihood of project success.
- Risk planning involves the following steps:
 1. Risk identification: This is the process of finding out the potential sources of risk that may affect the project objectives, such as scope, schedule, cost, quality, etc. Risk identification can be done using various techniques, such as brainstorming, checklists, interviews, surveys,

historical data, etc.

2. Risk analysis: This is the process of assessing the probability and impact of each identified risk on the project objectives. Risk analysis can be done using qualitative or quantitative methods, such as risk matrices, risk scoring, decision trees, simulation, etc.
 3. Risk response: This is the process of developing strategies to avoid, mitigate, transfer, or accept the risks that have been analyzed. Risk response can be done using various techniques, such as contingency plans, risk reserves, risk contracts, risk audits, etc.
 4. Risk monitoring and control: This is the process of tracking, reviewing, and updating the risk status and the effectiveness of the risk responses throughout the project lifecycle. Risk monitoring and control can be done using various techniques, such as risk registers, risk reports, risk reviews, risk metrics, etc.
- Some of the benefits of risk planning in spm are:
 - It helps to identify and prioritize the most critical risks that may affect the project success.
 - It helps to allocate the appropriate resources and time to deal with the risks.
 - It helps to improve the communication and collaboration among the project stakeholders regarding the risks and their responses.
 - It helps to enhance the quality and reliability of the software product and the customer satisfaction.
 - It helps to avoid or minimize the negative impacts of the risks and exploit or maximize the positive opportunities.
 - Some of the challenges of risk planning in spm are:
 - It may be difficult to identify all the possible risks that may occur in a complex and dynamic software project environment.
 - It may be difficult to estimate the probability and impact of the risks accurately and objectively.
 - It may be difficult to implement the risk responses effectively and efficiently, especially when the risks are interrelated or change frequently.
 - It may be difficult to monitor and control the risks continuously and consistently, especially when the project scope, schedule, or cost changes.
 - Some of the mnemonics and learning tricks for risk planning in spm are:
 - A mnemonic to remember the four steps of risk planning is: I Am Ready to Monitor (IAR2M).
 - A mnemonic to remember the four types of risk responses is: Avoid, Mitigate, Transfer, Accept (AMTA).
 - A learning trick to understand the difference between qualitative and quantitative risk analysis is: Qualitative is about Quality (subjective), quantitative is about Quantity (objective).

- tive), Quantitative is about Quantity (objective).
- A learning trick to understand the difference between risk and uncertainty is: Risk is known unknowns, Uncertainty is unknown unknowns.

Risk Management in spm

- Risk management is a sequence of steps that help a software team to understand, analyze and manage uncertainty.
- Risk management consists of four main phases: risk identification, risk analysis, risk planning and risk monitoring.
- Risk identification is the process of finding out the potential sources of risk that may affect the software project, such as requirements, technology, resources, stakeholders, etc.
- Risk analysis is the process of estimating the probability and impact of each identified risk, and prioritizing them according to their severity.
- Risk planning is the process of developing strategies to avoid, reduce, transfer or accept the risks, and allocating resources and responsibilities for implementing them.
- Risk monitoring is the process of tracking the status of the risks and the effectiveness of the risk plans, and updating them as needed.
- Risk management is one of the core tents of the philosophy of Software Project Management (SPM), as it helps to ensure the quality, cost and schedule of the software project.
- Risk management is also a system-wide managerial tool to analyze and manage safety and security risks to personnel, assets and operations, as in the case of the United Nations Security Management System (UNSMS).

Some mnemonics and learning tricks for risk management in spm are:

- To remember the four phases of risk management, use the acronym **IAPM** (Identification, Analysis, Planning, Monitoring).
- To remember the four types of risk strategies, use the acronym **ARTA** (Avoid, Reduce, Transfer, Accept).
- To remember the factors that affect the probability and impact of risks, use the acronym **TRUST** (Time, Resources, Users, Scope, Technology).
- To remember the components of a risk register, use the acronym **PIMP** (Probability, Impact, Mitigation, Plan).

PERT Technique in spm

- PERT stands for Program Evaluation and Review Technique. It is a project management planning tool used to calculate the amount of time it will take to realistically finish a project .
- PERT charts are used to plan tasks within a project, making it easier to schedule and coordinate team members. They also help in keeping track of the progress, or lack thereof, of the overall project .

- PERT was first developed by the US Navy SPO (Special Projects Office) in 1967 during the Polaris missile development program then it was applied to the other industries. Both CPM and PERT are complementary tools and they are developed at roughly the same time .
- PERT is based on the concept of a network diagram, which is a graphical representation of the project activities and their interdependencies. A network diagram consists of nodes and arrows. Nodes represent the activities or tasks, and arrows represent the precedence relationships or dependencies among them.
- PERT uses a three-point estimating technique to calculate the expected time for each activity. The three points are: optimistic time (O), pessimistic time (P), and most likely time (M). The expected time (E) is calculated as follows :

$$E = (O + 4M + P) / 6$$

- PERT also calculates the variance (V) and the standard deviation (SD) of each activity, which are measures of the uncertainty or risk involved in the time estimates. The variance is calculated as follows :

$$V = ((P - O) / 6)^2$$

$$SD = \sqrt{V}$$

- PERT uses the expected time, variance, and standard deviation of each activity to determine the critical path of the project, which is the longest path of activities in the network diagram that determines the minimum duration of the project. The critical path is identified by adding the expected times of the activities along each path and finding the path with the maximum total time .
- PERT also calculates the earliest start time (ES), earliest finish time (EF), latest start time (LS), and latest finish time (LF) of each activity, which are the time windows within which the activity can be performed without affecting the project duration. These are calculated as follows :

$$ES = \text{maximum EF of all immediate predecessors}$$

$$EF = ES + E$$

$$LF = \text{minimum LS of all immediate successors}$$

$$LS = LF - E$$

- PERT also calculates the slack or float of each activity, which is the amount of time that the activity can be delayed or advanced without affecting the project duration. There are two types of slack: total slack and free slack. Total slack is the difference between the latest and earliest times of the activity. Free slack is the difference between the earliest finish time of the

activity and the earliest start time of the next activity. Slack is calculated as follows :

$$\text{Total slack} = \text{LF} - \text{EF} = \text{LS} - \text{ES}$$

$$\text{Free slack} = \text{minimum ES of all immediate successors} - \text{EF}$$

- PERT can be used to analyze the project work schedule by focusing on each task and calculate the minimum time required to complete the project. It can also be used to identify the critical activities that need more attention and resources, and the non-critical activities that have some flexibility and buffer. PERT can also be used to evaluate the impact of changes or delays in the project activities on the overall project duration .
- Some advantages of PERT are :
 - It helps in planning and controlling complex and uncertain projects.
 - It provides a visual and logical representation of the project activities and their dependencies.
 - It allows for the incorporation of uncertainty and risk in the time estimates.
 - It helps in identifying the critical path and the critical activities that determine the project duration.
 - It helps in optimizing the use of resources and time.
 - It helps in monitoring and tracking the project progress and performance.
- Some disadvantages of PERT are :
 - It can be time-consuming and costly to collect and analyze the data for the three-point estimates.
 - It can be difficult to estimate the optimistic, pessimistic, and most likely

Monte Carlo Simulation in spm

- Monte Carlo simulation is a statistical technique that uses random sampling to generate possible outcomes of a process or system that involves uncertainty or variability.
- spm is a software package for statistical parametric mapping, which is a method for analyzing brain imaging data, such as functional magnetic resonance imaging (fMRI) or positron emission tomography (PET).
- Monte Carlo simulation can be used in spm for various purposes, such as:
 - Estimating the significance of activation maps by comparing them to null distributions generated by random permutations or bootstrapping.
 - Evaluating the performance of different statistical tests or models by simulating data with known properties and comparing the results to the true values.

- Assessing the sensitivity and specificity of different thresholds or corrections for multiple comparisons by simulating data with different levels of noise and signal.
- Exploring the effects of different experimental designs or parameters on the statistical power or validity of the analysis by simulating data with different scenarios or conditions.
- Some advantages of using Monte Carlo simulation in spm are:
 - It can provide a more accurate and realistic assessment of the uncertainty and variability in the data and the analysis than analytical methods or assumptions.
 - It can handle complex and nonlinear systems or processes that may not have analytical solutions or closed-form expressions.
 - It can be easily implemented and customized using existing tools and functions in spm or other software packages, such as MATLAB or R.
- Some disadvantages or limitations of using Monte Carlo simulation in spm are:
 - It can be computationally intensive and time-consuming, especially for large or high-dimensional data sets or models, or for a large number of simulations or iterations.
 - It can be affected by the quality and quantity of the random samples or draws, which may introduce bias or error if they are not representative or sufficient.
 - It can be difficult to interpret or generalize the results of the simulation, as they may depend on the specific choices or assumptions made for the simulation, such as the distribution, parameters, or model of the data or the system.
- An example of using Monte Carlo simulation in spm is the nonparametric permutation test, which is a method for testing the null hypothesis that there is no difference between two or more groups or conditions in the brain activation patterns. The steps of the permutation test are:
 - For each subject, compute the contrast image that represents the difference between the groups or conditions of interest, such as the activation map for task A minus task B.
 - Randomly shuffle or permute the labels of the groups or conditions for each subject, such as assigning task A or task B to each subject by chance, and compute the contrast image for each permutation.
 - Repeat the permutation step for a large number of times, such as 10,000 times, and store the contrast images for each permutation.
 - For each voxel or region in the brain, compare the original contrast image to the distribution of the permuted contrast images, and compute the p-value or the proportion of the permuted contrast images that are more extreme than the original contrast image.
 - Apply a threshold or a correction for multiple comparisons to the p-values, and identify the voxels or regions that show a significant difference between the groups or conditions.

Resource Allocation in SPM

- Resource allocation in SPM (software project management) is the process of assigning and scheduling available resources in the most effective and economical way possible .
- Resources include staff, budgets, materials, and equipment needed for the successful completion of a project.
- Resource allocation is important because it gives a clear picture of the amount of work that has to be done, the time and cost required, and the progress and performance of the project team .
- Resource allocation involves the following steps:
 - Divide the project into tasks and subtasks
 - Estimate the effort, duration, and cost of each task
 - Identify the dependencies and constraints among tasks
 - Determine the availability and capacity of each resource
 - Assign resources to tasks based on their skills, preferences, and priorities
 - Monitor and control the resource utilization and allocation throughout the project lifecycle
- Resource allocation can be done using various techniques and tools, such as :
 - Resource leveling: balancing the demand and supply of resources by adjusting the project schedule
 - Resource smoothing: minimizing the fluctuations in resource usage by shifting or splitting tasks within the project slack
 - Resource histograms: graphical representations of the resource allocation over time
 - Resource calendars: tables or charts that show the availability and working hours of each resource
 - Resource management software: applications that help plan, track, and optimize the resource allocation
- Resource allocation can be challenging due to factors such as :
 - Resource scarcity: the limited availability of resources in terms of quantity, quality, or location
 - Resource conflict: the competition or disagreement among stakeholders over the allocation of resources
 - Resource uncertainty: the unpredictability or variability of resource availability, demand, or performance
 - Resource change: the modification or replacement of resources due to changes in scope, schedule, or quality
- Resource allocation can be improved by following best practices such as :
 - Define the project scope and objectives clearly and realistically
 - Involve the project team and stakeholders in the resource planning and allocation process
 - Use data and evidence to support the resource estimation and allocation decisions

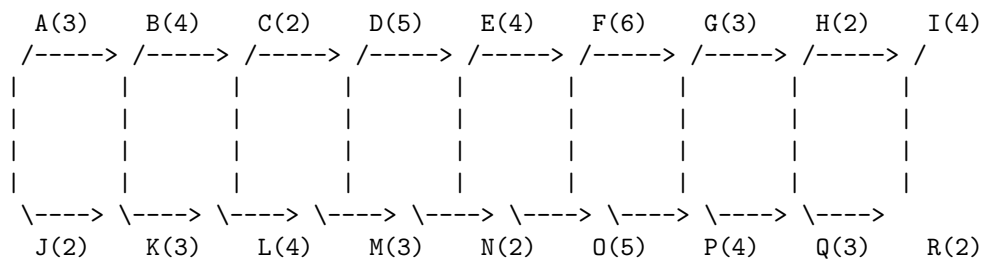
- Communicate the resource allocation plan and expectations to all parties
- Review and update the resource allocation regularly and proactively
- Resolve resource issues and conflicts promptly and collaboratively

Creation of Critical Paths in spm

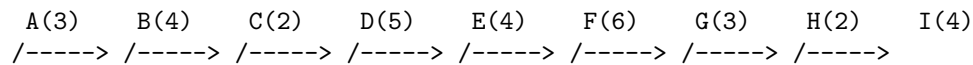
- Critical path is the longest sequence of activities from project start to finish that must be completed to ensure the project is finished by a certain time .
- Critical path method (CPM) is an algorithm for planning, managing and analyzing the timing of a project.
- The steps to create a critical path in spm are:
 - Step 1: Activity specification. Identify all the activities required to complete the project and list them in a table with their durations and dependencies.
 - Step 2: Activity sequence establishment. Draw a network diagram that shows the logical order of the activities and their dependencies using nodes and arrows. The nodes represent the activities and the arrows represent the precedence relationships.
 - Step 3: Network diagram. Label each node with the activity name, duration, earliest start time (ES), earliest finish time (EF), latest start time (LS), and latest finish time (LF). The ES and EF are calculated by adding the duration to the ES or EF of the preceding activity. The LS and LF are calculated by subtracting the duration from the LS or LF of the succeeding activity.
 - Step 4: Estimates for each activity. Estimate the optimistic time (O), most likely time (M), and pessimistic time (P) for each activity using historical data, expert judgment, or other methods. The expected time (E) for each activity can be calculated using the formula: $E = (O + 4M + P) / 6$. The variance (V) for each activity can be calculated using the formula: $V = ((P - O) / 6)^2$.
 - Step 5: Identification of the critical path. The critical path is the path with the longest duration in the network diagram. It can be identified by finding the activities with zero slack, which is the difference between the LF and EF or the LS and ES. The activities on the critical path have the highest risk of delaying the project completion and must be closely monitored and controlled.
 - Step 6: Critical path diagram to show project progresses. The critical path diagram is a graphical representation of the project schedule that shows the critical path and the progress of each activity. It can be updated periodically to reflect the actual start and finish times of the activities and to identify any deviations from the planned schedule.

- A mnemonic to remember the steps to create a critical path in spm is: **AS END** (Activity Specification, Activity Sequence Establishment, Network Diagram, Estimates, Critical Path Identification, Critical Path Diagram).
- A learning trick to understand the concept of critical path is to imagine a chain of dominoes. The dominoes represent the activities and the order in which they fall represents the precedence relationships. The critical path is the longest chain of dominoes that must fall to complete the project. If any domino on the critical path is delayed, the whole project will be delayed. The dominoes that are not on the critical path have some slack, which means they can fall earlier or later without affecting the project completion.
- An example of a network diagram and a critical path diagram for a software project is shown below:

Network Diagram:



Critical Path Diagram:



Cost Schedules in spm

- Cost schedules are tools that help project managers to estimate, monitor, and control the costs of a project.
- Cost schedules are usually based on the work breakdown structure (WBS) of the project, which divides the project into manageable tasks and assigns them cost estimates and durations.
- Cost schedules can be presented in different formats, such as tables, charts, graphs, or histograms, depending on the level of detail and the purpose of the communication.
- Cost schedules can help project managers to:
 - Plan and allocate the project budget and resources.
 - Track and measure the actual costs and variances against the planned costs and baselines.
 - Forecast the future costs and cash flows of the project.

- Identify and mitigate the risks and issues related to the project costs.
 - Report and communicate the project cost performance and status to the stakeholders.
- Cost schedules can be created and updated using various methods and techniques, such as:
 - Analogous estimating: using the historical data and experience from similar projects to estimate the costs of the current project.
 - Parametric estimating: using statistical models and formulas to calculate the costs based on the project parameters and variables.
 - Bottom-up estimating: aggregating the costs of the lower-level tasks and components to estimate the costs of the higher-level deliverables and the whole project.
 - Three-point estimating: using the optimistic, pessimistic, and most likely estimates to calculate the expected costs and the range of uncertainty.
 - Reserve analysis: adding contingency reserves and management reserves to the cost estimates to account for the known and unknown risks and changes.
- Cost schedules can be influenced by various factors and constraints, such as:
 - Scope: the size, complexity, and quality of the project deliverables and requirements.
 - Time: the duration, sequence, and dependencies of the project activities and milestones.
 - Resources: the availability, quantity, and quality of the human, material, and equipment resources needed for the project.
 - Quality: the standards, criteria, and specifications that the project deliverables and processes must meet or exceed.
 - Risks: the uncertainties, threats, and opportunities that may affect the project costs positively or negatively.
 - Stakeholders: the expectations, interests, and influence of the people and organizations involved in or affected by the project.
 - Changes: the modifications, additions, or deletions of the project scope, time, resources, quality, risks, or stakeholders that may impact the project costs.
- Cost schedules can be evaluated and controlled using various tools and techniques, such as:
 - Earned value management (EVM): a method that compares the planned value, earned value, and actual cost of the project to measure the cost variance and the cost performance index.
 - Trend analysis: a technique that examines the patterns and changes of the project costs over time to identify the trends and forecast the future costs.
 - Variance analysis: a technique that analyzes the differences and deviations between the planned and actual costs to identify the causes and effects of the variances.

- Performance reviews: a process that assesses and reports the project cost performance and status to the stakeholders using various metrics and indicators.
- Change control: a process that manages and approves the changes to the project cost baselines and budgets using a change request system.

Unit - 4 - Project Management and Control for the notes of the course Software Project Management in spm

- Project management and control is the process of monitoring and controlling the project activities to ensure that they meet the project objectives and deliverables.
- Project management and control involves the following aspects:
 - Creating a framework for management and control, which defines the roles, responsibilities, communication channels, reporting mechanisms, and tools for the project team and stakeholders.
 - Collecting the data, which involves gathering and analyzing the information about the project progress, performance, quality, risks, issues, and changes.
 - Visualizing the progress, which involves presenting the data in a clear and concise way, using charts, graphs, dashboards, and reports, to show the project status, trends, and deviations.
 - Cost monitoring, which involves tracking and controlling the project budget, expenses, and cash flow, and comparing them with the baseline and forecast values.
 - Earned value analysis, which is a technique that integrates the scope, time, and cost aspects of the project, and measures the project performance in terms of value earned, schedule variance, and cost variance.
 - Prioritizing monitoring, which involves focusing on the most critical and important aspects of the project, such as the key deliverables, milestones, risks, and issues, and allocating the resources and attention accordingly.
 - Getting the project back to target, which involves identifying and implementing corrective and preventive actions to address the project deviations, problems, and changes, and ensuring that the project is aligned with the objectives and expectations.
 - Change control, which is a process that ensures that any changes to the project scope, schedule, cost, quality, or resources are formally requested, evaluated, approved, and implemented, and that their impacts are assessed and communicated.
 - Software configuration management, which is a process that ensures that the software products and artifacts are identified, controlled, verified, and audited, and that their versions, baselines, and changes are tracked and documented.
 - Contract management, which involves managing the contractual relationships and obligations between the project and the external par-

ties, such as the customers, suppliers, and subcontractors, and ensuring that the contract terms and conditions are fulfilled and the risks are minimized.

- Some mnemonics and learning tricks for the unit are:
 - The acronym **PVC** can be used to remember the three aspects of earned value analysis: **P**ercent complete, **V**alue earned, and **C**ost variance.
 - The acronym **CRUD** can be used to remember the four steps of change control: **C**hange request, **R**evue and approval, **U**date and implementation, and **D**ocumentation and communication.
 - The acronym **SMART** can be used to remember the characteristics of good project objectives and deliverables: **S**pecific, **M**easurable, **A**chievable, **R**elevant, and **T**ime-bound.

Framework for Management and Control in spm

- Framework for Management and Control in spm is a set of processes and tools that help software project managers to monitor and control the project activities and outcomes.
- The main objectives of the framework are to ensure that the project meets its goals, stays on budget, and delivers quality software products to the customers.
- The framework consists of four main components :
 - Creating Framework: This involves defining the project scope, objectives, deliverables, milestones, tasks, resources, risks, and quality standards. It also involves establishing the project organization, communication, and reporting mechanisms.
 - Collecting Data: This involves gathering and recording the project data, such as the actual work done, the actual cost incurred, the actual quality achieved, and the actual issues and changes encountered. The data can be collected from various sources, such as project team members, customers, stakeholders, and tools.
 - Visualizing Progress: This involves analyzing and presenting the project data in a clear and concise way, such as using charts, graphs, tables, dashboards, and reports. The visualization helps to compare the actual project performance with the planned or expected performance, and to identify any deviations, trends, or patterns.
 - Cost Monitoring: This involves tracking and controlling the project cost, such as the budget, the expenses, the revenue, and the profit. It also involves using techniques such as earned value management (EVM) to measure the project value and progress in terms of cost and schedule.
 - Prioritizing Monitoring: This involves focusing on the most critical and important aspects of the project, such as the scope, the quality, the risks, and the customer satisfaction. It also involves using techniques such as risk management, quality management, and stake-

holder management to manage and mitigate the potential threats and issues that may affect the project success.

- Getting Project Back to Target: This involves taking corrective and preventive actions to address the project problems and deviations, and to bring the project back on track. It also involves using techniques such as change management, configuration management, and issue management to handle and document the changes and issues that may arise during the project execution.
- Change Control: This involves managing and controlling the changes that may occur in the project scope, requirements, design, implementation, testing, deployment, or maintenance. It also involves using techniques such as change request, change impact analysis, change approval, change implementation, and change verification to ensure the changes are justified, authorized, implemented, and verified.
- Managing Contracts: This involves managing and controlling the contracts that may exist between the project and the external parties, such as the customers, the suppliers, the subcontractors, or the partners. It also involves using techniques such as contract types, contract stages, contract terms, and contract administration to ensure the contracts are clear, fair, and compliant.
- A possible mnemonic to remember the components of the framework is **CCVP CGCM** (Creating, Collecting, Visualizing, Prioritizing, Cost, Getting, Change, Managing).
- A possible learning trick to understand the framework is to use a project management software tool, such as Microsoft Project, that can help to create, collect, visualize, and monitor the project data and activities. The tool can also help to perform cost, risk, quality, change, and contract management functions.

Collection of Data in spm

- Collection of data is the process of gathering and measuring information on variables of interest, in an established systematic fashion that enables one to answer stated research questions, test hypotheses, and evaluate outcomes.
- Data can be collected from primary or secondary sources.
- Primary data is the data that is collected directly from the original source for the purpose of a specific research study. Examples of primary data are surveys, interviews, observations, experiments, etc.
- Secondary data is the data that is collected from other sources that have already been published or processed for some other purpose. Examples of secondary data are books, journals, reports, websites, etc.
- The advantages of primary data are:
 - It is more accurate and reliable as it is collected by the researcher himself/herself.
 - It is more relevant and specific to the research problem and objectives.

- It is more timely and up-to-date as it reflects the current situation.
- The disadvantages of primary data are:
 - It is more costly and time-consuming as it involves designing and conducting the data collection process.
 - It is more difficult and complex as it requires specialized skills and techniques.
 - It is more prone to errors and biases as it depends on the quality and validity of the data collection instruments and methods.
- The advantages of secondary data are:
 - It is more economical and convenient as it is readily available and accessible.
 - It is more comprehensive and extensive as it covers a large scope and variety of data.
 - It is more objective and credible as it is collected and verified by experts and authorities.
- The disadvantages of secondary data are:
 - It is less accurate and reliable as it may be outdated, incomplete, or inaccurate.
 - It is less relevant and specific as it may not suit the research problem and objectives.
 - It is less flexible and adaptable as it may not fit the data analysis and interpretation needs of the researcher.
- The steps involved in the collection of data are:
 - Define the research problem and objectives.
 - Identify the data sources and types.
 - Select the data collection method and instrument.
 - Design the data collection plan and procedure.
 - Execute the data collection process.
 - Validate and verify the data collected.
 - Prepare the data for analysis and interpretation.

Visualizing Progress in spm

- spm stands for Software Project Management, which is the process of planning, organizing, monitoring, and controlling software projects.
- Visualizing progress in spm is the practice of using various tools and techniques to track and communicate the status and performance of software projects to stakeholders and team members.
- Some of the benefits of visualizing progress in spm are:
 - It helps to identify and resolve issues and risks early.
 - It increases transparency and accountability among project participants.
 - It motivates and engages team members by showing them their achievements and challenges.
 - It facilitates collaboration and feedback among project participants.
 - It supports decision making and problem solving by providing rele-

vant and timely information.

- Some of the common tools and techniques for visualizing progress in spm are:
 - Gantt charts: A graphical representation of the project schedule, showing the start and end dates, dependencies, and milestones of each task or activity.
 - Burn-down charts: A graphical representation of the remaining work or effort to be completed in a given time period, showing the progress and deviation from the planned scope or budget.
 - Kanban boards: A visual system for managing work in progress, showing the status and flow of tasks or items across different stages or columns, such as to-do, in progress, and done.
 - Dashboards: A graphical display of key performance indicators (KPIs) or metrics, such as quality, cost, time, scope, and customer satisfaction, showing the current and historical values and trends.
 - Reports: A written or oral presentation of the project information, such as status, issues, risks, changes, and lessons learned, to inform and update the project stakeholders and team members.

Cost Monitoring in spm

- Cost monitoring in Software Performance Management (SPM) is a process or function that monitors the expenses related to a software project, including hardware, development, maintenance and operations.
- Cost monitoring helps project managers to track progress and make sure they stay within budget. It also helps to identify and control any deviations or risks that may affect the project's cost performance.
- Cost monitoring involves collecting and analyzing cost data, such as planned costs, actual costs, and earned value, to measure the project's cost performance and forecast the project's cost at completion .
- Cost monitoring can be done using various tools and techniques, such as cost reports, cost variance analysis, cost performance index, cost baseline, and earned value analysis .
- Cost monitoring can provide several benefits for software projects, such as:
 - Improving the accuracy and reliability of cost estimates and budgets.
 - Enhancing the visibility and accountability of project costs and resources.
 - Supporting the decision-making and problem-solving processes of project managers and stakeholders.
 - Increasing the efficiency and effectiveness of project execution and delivery.
 - Reducing the risks of cost overruns, delays, and quality issues.
- A possible mnemonic to remember the benefits of cost monitoring is **IERSR** (Improve, Enhance, Support, Increase, Reduce).
- A possible learning trick to understand the concept of earned value analy-

sis is to use the analogy of a pizza delivery. Suppose you order a pizza that costs \$10 and is expected to be delivered in 30 minutes. The planned value (PV) of the pizza is \$10, the actual cost (AC) of the pizza is the amount you pay for it, and the earned value (EV) of the pizza is the value of the pizza that has been delivered to you. For example, if the pizza arrives in 20 minutes and you pay \$12 for it, then the AC is \$12 and the EV is \$10. If the pizza arrives in 40 minutes and you pay \$8 for it, then the AC is \$8 and the EV is \$10. If the pizza arrives in 30 minutes and you pay \$10 for it, then the AC is \$10 and the EV is \$10. The cost variance (CV) is the difference between the EV and the AC, and the cost performance index (CPI) is the ratio of the EV to the AC. For example, if the pizza arrives in 20 minutes and you pay \$12 for it, then the CV is -\$2 and the CPI is 0.83. If the pizza arrives in 40 minutes and you pay \$8 for it, then the CV is \$2 and the CPI is 1.25. If the pizza arrives in 30 minutes and you pay \$10 for it, then the CV is \$0 and the CPI is 1. The CV and the CPI indicate whether the project is under budget or over budget. A negative CV or a CPI less than 1 means the project is over budget, while a positive CV or a CPI greater than 1 means the project is under budget. A zero CV or a CPI equal to 1 means the project is on budget.

Earned Value Analysis in spm

- Earned Value Analysis (EVA) is a project management technique that allows to measure the performance and progress of a project based on the planned and actual costs and the work done .
- EVA uses three key metrics to track the project status: Planned Value (PV), Earned Value (EV), and Actual Cost (AC) .
 - PV is the budgeted cost of the work scheduled to be done by a certain date . It is also known as Budgeted Cost of Work Scheduled (BCWS) .
 - EV is the budgeted cost of the work actually completed by a certain date . It is also known as Budgeted Cost of Work Performed (BCWP) .
 - AC is the actual cost incurred for the work done by a certain date . It is also known as Actual Cost of Work Performed (ACWP) .
- EVA uses these metrics to calculate the following indicators of project performance :
 - Schedule Variance (SV) = EV - PV. It shows whether the project is ahead or behind schedule . A positive SV means the project is ahead of schedule, while a negative SV means the project is behind schedule .
 - Cost Variance (CV) = EV - AC. It shows whether the project is under or over budget . A positive CV means the project is under budget, while a negative CV means the project is over budget .
 - Schedule Performance Index (SPI) = EV / PV. It shows how efficiently the project is using time . An SPI of 1 means the project is

on schedule, while an SPI greater than 1 means the project is ahead of schedule, and an SPI less than 1 means the project is behind schedule .

- Cost Performance Index (CPI) = EV / AC . It shows how efficiently the project is using resources . A CPI of 1 means the project is on budget, while a CPI greater than 1 means the project is under budget, and a CPI less than 1 means the project is over budget .
- Estimate at Completion (EAC) = BAC / CPI . It shows the estimated total cost of the project at completion . BAC is the original budget at completion .
- Estimate to Complete (ETC) = $EAC - AC$. It shows the estimated remaining cost to complete the project .
- Variance at Completion (VAC) = $BAC - EAC$. It shows the expected cost overrun or underrun at the end of the project .
- EVA can help the project manager to monitor and control the project by providing an integrated view of the project's scope, time, and cost . It can also help to identify and correct any deviations from the plan, forecast the project outcome, and communicate the project status to the stakeholders .
- EVA requires the project to have a well-defined work breakdown structure (WBS), a realistic and detailed schedule, and a reliable and accurate cost estimation . It also requires the project to have a clear baseline, a regular and consistent measurement of progress, and a timely and accurate reporting of costs [^3

Prioritizing Monitoring in spm

- Prioritizing monitoring in software project management (spm) is the process of deciding which aspects of the project need to be monitored more closely and frequently than others, based on their importance, risk, and complexity.
- Prioritizing monitoring helps to allocate the limited resources and time of the project manager and the team to the most critical and relevant areas of the project, and to avoid wasting effort on less important or trivial issues.
- Prioritizing monitoring also helps to identify and address potential problems and deviations from the project plan before they escalate and affect the project outcomes and quality.
- Some of the factors that can be used to prioritize monitoring in spm are:
 - The project objectives and scope: The project manager should monitor the progress and performance of the project against the predefined objectives and scope, and ensure that the project deliverables meet the expectations and requirements of the stakeholders and customers.

- The project risks and uncertainties: The project manager should monitor the sources and impacts of the project risks and uncertainties, and implement the appropriate risk management strategies and contingency plans to mitigate or avoid them.
 - The project dependencies and constraints: The project manager should monitor the dependencies and constraints that affect the project schedule, budget, resources, and quality, and coordinate with the relevant parties to resolve any conflicts or issues that arise.
 - The project milestones and deliverables: The project manager should monitor the completion and quality of the project milestones and deliverables, and ensure that they are delivered on time and within the agreed scope and budget.
 - The project stakeholders and communication: The project manager should monitor the expectations and satisfaction of the project stakeholders and customers, and maintain effective and timely communication with them throughout the project lifecycle.
 - The project team and performance: The project manager should monitor the performance and productivity of the project team, and provide them with the necessary guidance, feedback, motivation, and recognition.
- A possible mnemonic to remember these factors is: **O.R.D.E.R.** (Objectives, Risks, Dependencies, Milestones, Communication)
 - A possible learning trick to apply these factors is to use a prioritization matrix, such as the Eisenhower matrix, to classify the monitoring tasks into four categories based on their urgency and importance:
 - Urgent and important: These are the tasks that need to be monitored and done immediately, as they have a high impact on the project success and quality. For example, resolving a major risk or issue that threatens the project objectives or scope.
 - Important but not urgent: These are the tasks that need to be monitored and planned for the near future, as they have a significant impact on the project success and quality. For example, completing a key milestone or deliverable that is due soon.
 - Urgent but not important: These are the tasks that need to be monitored and delegated or outsourced, as they have a low impact on the project success and quality. For example, responding to a minor stakeholder request or query that is time-sensitive.
 - Not urgent and not important: These are the tasks that need to be monitored and eliminated or minimized, as they have a negligible impact on the project success and quality. For example, updating a trivial document or report that is not required or used.
 - A possible ascii diagram to illustrate the prioritization matrix is:

+-----+-----+

	Urgent
	Do (High)
Important	Plan (Medium)
	Delegate (Low)
Not Important	Eliminate (None)

Project Tracking in spm

- Project tracking is the process of monitoring and controlling the progress, quality, and performance of a software project.
- Project tracking involves collecting, analyzing, and reporting data on the project status, issues, risks, changes, and deliverables.
- Project tracking helps to ensure that the project meets the objectives, scope, schedule, budget, and quality standards agreed with the stakeholders.
- Project tracking also helps to identify and resolve problems, communicate with the project team and stakeholders, and implement corrective actions as needed.
- Project tracking can be done using various tools and techniques, such as:
 - Project management software: This is a software application that helps to plan, execute, monitor, and control software projects. It can provide features such as task management, resource allocation, scheduling, cost estimation, risk management, change management, document management, and reporting. Some examples of project management software are Microsoft Project, Jira, Trello, and Asana.
 - Project dashboard: This is a graphical representation of the key performance indicators (KPIs) and metrics of a software project. It can provide a quick and easy way to visualize the project status, progress, health, and performance. It can also highlight the issues, risks, and changes that need attention. Some examples of project dashboard tools are Power BI, Tableau, and Smartsheet.
 - Project review: This is a formal or informal meeting that involves the project team and stakeholders to discuss the project progress, issues, risks, changes, and deliverables. It can provide an opportunity

to share information, feedback, and lessons learned, and to make decisions and actions for the next phase or iteration of the project. Some examples of project review methods are stand-up meetings, sprint reviews, and retrospectives.

- Project audit: This is an independent and objective examination of the project processes, products, and results. It can provide an assessment of the project compliance, quality, and performance, and identify the strengths, weaknesses, opportunities, and threats of the project. It can also provide recommendations and best practices for improving the project management and delivery. Some examples of project audit standards are ISO 9001, CMMI, and PMBOK.

Change Control in spm

- Change control is the process of managing and documenting any changes that occur during the life cycle of a project or a system.
- Change control aims to ensure that the changes are beneficial, feasible, consistent, and aligned with the project or system objectives.
- Change control in spm (strategic portfolio management) involves the following steps:
 - Propose change: A change request (CR) is submitted by anyone who identifies a need for change, such as a stakeholder, a customer, or a team member.
 - Impact summary: The CR is evaluated to assess its technical merit, potential side effects, overall impact on other configuration objects and system functions, and the projected cost of the change.
 - Making a decision: The CR is approved, rejected, or deferred by the change control board (CCB), which is a group of authorized decision-makers.
 - Make the change: If the CR is approved, the change is implemented according to the agreed plan and schedule.
 - Closure: The change is verified, tested, and documented, and the CR is closed.
- Change control in spm helps to visualize strategies and targets, know whether the objectives are met, and make decisions with confidence.
- Change control in spm also helps to respond quickly when changes are needed, prioritize and schedule the right work, align teams, drive efficiency, and stay on strategy.
- A possible mnemonic to remember the steps of change control in spm is **PIMMC** (Propose, Impact, Make, Make, Closure).
- A possible learning trick to understand the concept of change control in spm is to compare it to a recipe. A recipe is a plan for making a dish, but sometimes you may need to change it based on the availability of ingredients, the preferences of the customers, or the feedback of the testers. Change control is the process of managing and documenting those changes, and making sure they are beneficial, feasible, consistent, and aligned with

the original recipe.

Software Configuration Management in spm

Software Configuration Management (SCM) is a process of controlling and managing the changes in software products during their development and maintenance. SCM aims to ensure the integrity, consistency, and quality of software products by identifying, tracking, and documenting the changes made to them.

Some of the main objectives of SCM are:

- To facilitate coordination and communication among developers, testers, and other stakeholders involved in software development.
- To maintain a history of changes made to software products and their versions, and to provide traceability and accountability for those changes.
- To prevent or minimize conflicts and errors caused by incompatible or inconsistent changes made by different developers or teams.
- To enable the reproduction and verification of software products and their components at any stage of development or deployment.
- To support the reuse and adaptation of software products and their components across different projects or contexts.

Some of the main activities of SCM are:

- Configuration identification: This involves defining and naming the software products and their components, and establishing their baselines and variants.
- Configuration control: This involves reviewing and approving the proposed changes to software products and their components, and implementing and recording those changes in a controlled manner.
- Configuration status accounting: This involves maintaining and reporting the status and history of software products and their components, and their relationships and dependencies.
- Configuration auditing: This involves verifying and validating that the software products and their components conform to their specifications and requirements, and that the changes made to them are authorized and documented.
- Configuration management planning: This involves defining and documenting the policies, procedures, tools, and roles and responsibilities for performing SCM activities.

Some of the main benefits of SCM are:

- It improves the quality and reliability of software products by reducing errors, defects, and failures caused by uncontrolled or inconsistent changes.
- It enhances the productivity and efficiency of software development by facilitating collaboration and coordination among developers and other stakeholders, and by automating and streamlining the SCM activities.

- It supports the flexibility and adaptability of software development by enabling the creation and management of different versions and variants of software products and their components, and by supporting the reuse and customization of them.
- It facilitates the delivery and deployment of software products by enabling the identification and retrieval of the correct and consistent versions and components of them, and by supporting the testing and verification of them.

Some of the main challenges of SCM are:

- It requires a significant amount of planning, coordination, and documentation to establish and maintain an effective SCM process and system.
- It involves a trade-off between the level of control and the level of flexibility in managing the changes to software products and their components, and finding the optimal balance can be difficult and context-dependent.
- It depends on the availability and suitability of the tools and technologies that support the SCM activities, and the integration and compatibility of them with the software development environment and process.
- It requires the commitment and cooperation of all the stakeholders involved in software development, and the alignment of their goals and expectations with the SCM objectives and policies.

Managing Contracts in spm

- SPM stands for Supplier Performance Management, which is a process of measuring, analyzing, and managing the performance of the suppliers that provide goods and services to an organization.
- Managing contracts in spm involves the following steps:
 - Define the scope, objectives, and deliverables of the contract, and establish the roles and responsibilities of the parties involved.
 - Select the appropriate type of contract based on the nature, complexity, and risk of the project, such as fixed price, cost reimbursable, or time and material contracts .
 - Negotiate the terms and conditions of the contract, such as the payment schedule, quality standards, performance indicators, and dispute resolution mechanisms.
 - Monitor and evaluate the performance of the supplier against the contract requirements, using tools such as scorecards, dashboards, and reports .
 - Communicate and collaborate with the supplier regularly, and provide feedback, recognition, and corrective actions as needed.
 - Manage any changes, issues, or risks that may arise during the contract execution, and document them accordingly.
 - Close the contract upon the completion or termination of the project, and conduct a lessons learned review to identify the best practices and areas for improvement.

- Some of the benefits of managing contracts in spm are:
 - Improved quality, efficiency, and reliability of the goods and services delivered by the supplier.
 - Reduced costs, risks, and delays associated with the contract execution.
 - Enhanced customer satisfaction and loyalty with the supplier's performance.
 - Strengthened relationship and trust between the organization and the supplier.
 - Increased innovation and competitiveness of the organization and the supplier.
- Some of the challenges of managing contracts in spm are:
 - Lack of clear and measurable contract requirements and expectations.
 - Difficulty in selecting and evaluating the right supplier for the project.
 - Inadequate communication and coordination between the organization and the supplier.
 - Resistance to change or feedback from the supplier or the organization.
 - Conflicts or disputes arising from the contract interpretation or implementation.
- A mnemonic to remember the steps of managing contracts in spm is: **Define, Select, Negotiate, Monitor, Communicate, Manage, Close.**

Contract Management in spm

- Contract management is the process of managing the contracts between an organization and its suppliers, customers, partners, or employees.
- Contract management involves planning, negotiating, executing, monitoring, and closing contracts in a way that ensures the fulfillment of the contractual obligations and the achievement of the desired outcomes.
- Contract management is an important aspect of software project management (spm), as it helps to define the scope, quality, cost, and schedule of the software project, as well as the roles and responsibilities of the parties involved.
- Contract management also helps to mitigate the risks and resolve the disputes that may arise during the software project lifecycle.
- Contract management in spm can be divided into three phases: pre-award, award, and post-award.

Pre-award phase

- The pre-award phase involves the identification of the software project needs and requirements, the selection of the appropriate contract type and procurement method, the preparation of the request for proposal (RFP) or invitation for bid (IFB), and the evaluation of the proposals or bids submitted by the potential suppliers.

- The pre-award phase aims to ensure that the contract is aligned with the software project objectives and that the best value for money is obtained from the suppliers.
- Some of the common contract types in software project management are:
 - Fixed price contracts: The supplier agrees to deliver the software project for a fixed amount of money, regardless of the actual costs incurred. The supplier bears the risk of cost overruns, but also benefits from cost savings. Fixed price contracts are suitable for software projects with well-defined scope and requirements, and low uncertainty.
 - Cost reimbursable contracts: The supplier is paid for the actual costs incurred in delivering the software project, plus a fee or profit margin. The fee can be fixed, incentive-based, or award-based. The customer bears the risk of cost overruns, but also benefits from cost savings. Cost reimbursable contracts are suitable for software projects with unclear scope and requirements, and high uncertainty.
 - Time and material contracts: The supplier is paid for the actual hours of work and the materials used in delivering the software project, plus a fee or profit margin. The fee can be fixed, incentive-based, or award-based. The risk of cost overruns is shared between the customer and the supplier, depending on the terms and conditions of the contract. Time and material contracts are suitable for software projects with flexible scope and requirements, and moderate uncertainty.

Award phase

- The award phase involves the negotiation of the contract terms and conditions, the finalization of the contract documents, and the signing of the contract by the authorized parties.
- The award phase aims to ensure that the contract is legally binding and enforceable, and that the rights and obligations of the parties are clearly defined and understood.
- Some of the key elements of a software project contract are:
 - Scope of work: The description of the software project deliverables, specifications, standards, and acceptance criteria.
 - Schedule: The timeline of the software project activities, milestones, and deadlines.
 - Cost: The budget of the software project, including the payment terms and methods, and the cost control and reporting mechanisms.
 - Quality: The quality requirements and expectations of the software project, including the quality assurance and quality control processes, and the quality metrics and indicators.
 - Risk: The identification, analysis, and mitigation of the potential

risks and uncertainties that may affect the software project, including the risk allocation and sharing methods, and the contingency plans and reserves.

- Communication: The communication methods and channels, the frequency and format of the communication, and the roles and responsibilities of the communication stakeholders.
- Change: The change management processes and procedures, including the change request and approval mechanisms, and the change impact and control methods.
- Dispute: The dispute resolution processes and procedures, including the escalation and arbitration mechanisms, and the applicable laws and jurisdictions.

Post-award phase

- The post-award phase involves the execution, monitoring, and closing of the contract, as well as the evaluation and improvement of the contract performance and outcomes.
- The post-award phase aims to ensure that the contract is fulfilled according to the agreed terms and conditions, and that the software project is delivered on time, within budget, and with the required quality.
- Some of the key activities of the post-award phase are:
 - Contract administration: The management of the contract documents, records, and correspondence, as well as the compliance with the contract obligations and regulations.
 - Contract performance: The measurement and reporting of the contract progress and results, as well as the identification and implementation of

Unit - 5 - Staffing in Software Projects for the notes of the course Software Project Management in spm

Staffing in software projects is the process of selecting, assigning, and managing the human resources that will work on a software project. Staffing is a critical activity that affects the quality, cost, and schedule of the software project. Staffing involves the following steps:

- **Planning:** This step involves defining the staffing needs and requirements for the project, such as the number, skills, roles, and availability of the software developers. Planning also involves creating a staffing strategy that aligns with the project goals, scope, budget, and timeline. A staffing strategy may include the following aspects :
 - The type of software development methodology to use, such as agile, waterfall, or hybrid.

- The type of software development team structure to adopt, such as functional, cross-functional, or matrix.
- The type of software development outsourcing model to apply, such as onshore, offshore, nearshore, or hybrid.
- The type of software development hiring model to follow, such as permanent, contract, or freelance.
- The type of software development compensation model to offer, such as fixed, hourly, or performance-based.
- **Recruiting:** This step involves finding and attracting the best candidates for the software project, based on the staffing strategy and the job specifications. Recruiting may involve the following activities :
 - Posting job ads on various platforms, such as online job boards, social media, or referrals.
 - Screening and filtering the applicants based on their resumes, portfolios, or tests.
 - Conducting interviews and assessments to evaluate the candidates' technical and soft skills, fit, and motivation.
 - Negotiating and finalizing the terms and conditions of the employment contract, such as salary, benefits, or incentives.
- **Assigning:** This step involves allocating the selected software developers to the appropriate project activities, tasks, and roles, based on their skills, experience, and preferences. Assigning may involve the following activities :
 - Defining the roles and responsibilities of each software developer, such as project manager, analyst, designer, programmer, tester, or maintainer.
 - Establishing the communication and collaboration channels and tools among the software developers, such as email, chat, or video conferencing.
 - Setting the performance and quality standards and expectations for each software developer, such as deadlines, deliverables, or feedback.
 - Balancing the workload and the skill mix of the software developers, such as by using a constraint satisfaction or optimization-based approach .
- **Managing:** This step involves monitoring, evaluating, and improving the performance and satisfaction of the software developers throughout the project lifecycle. Managing may involve the following activities :
 - Providing regular feedback and recognition to the software developers, such as by using a performance appraisal or reward system.
 - Providing ongoing training and development opportunities to the software developers, such as by offering mentoring, coaching, or certification programs.
 - Providing adequate support and resources to the software developers, such as by resolving issues, conflicts, or challenges.
 - Retaining and motivating the software developers, such as by creating a positive and engaging work environment, culture, or values.

Some mnemonics and learning tricks for the unit are:

- Remember the four steps of staffing in software projects as **PRAM**: Planning, Recruiting, Assigning, and Managing.
- Remember the five aspects of a staffing strategy as **TTHHC**: Type of methodology, Type of team structure, Type of outsourcing model, Type of hiring model, and Type of compensation model.
- Remember the four activities of recruiting as **PSNC**: Posting, Screening, Negotiating, and Contracting.
- Remember the four activities of assigning as **DEBS**: Defining, Establishing, Setting, and Balancing.
- Remember the four activities of managing as **PERT**: Providing feedback, Providing training, Providing support, and Providing retention.

Managing People in spm

- spm stands for software project management, which is the discipline of planning, organizing, leading, and controlling software projects.
- Managing people is one of the key skills of a software project manager, as the human factor is the main cause of project success or failure.
- Managing people involves selecting the best people for the project, motivating them, organizing them into effective teams, and resolving any conflicts or issues that may arise.
- Some of the best practices for managing people in spm are:
 - Manage your own workload first, before delegating tasks to others. This will help you set realistic expectations, prioritize your activities, and avoid micromanaging your team members.
 - Communicate clearly and frequently with your team, stakeholders, and customers. Use appropriate channels and tools, such as email, chat, video conferencing, collaboration platforms, etc. to share information, feedback, and updates. Establish regular meetings and reports to monitor progress and address any problems or risks.
 - Recognize and reward the achievements and contributions of your team members. Provide positive and constructive feedback, praise, recognition, incentives, and opportunities for growth and development. This will help you build trust, loyalty, and motivation among your team.
 - Understand the different models of motivation and apply them to your team members. Some of the common models are:
 - * Maslow's hierarchy of needs, which suggests that people have five levels of needs: physiological, safety, social, esteem, and self-actualization. A higher level of need becomes a motivator only

when the lower level is satisfied.

- * Herzberg's two-factor theory, which distinguishes between hygiene factors and motivators. Hygiene factors are the basic conditions that prevent dissatisfaction, such as salary, working environment, policies, etc. Motivators are the factors that increase satisfaction, such as achievement, recognition, responsibility, etc.
 - * McClelland's theory of needs, which identifies three types of needs: achievement, affiliation, and power. Achievement-oriented people seek challenging goals, feedback, and recognition. Affiliation-oriented people seek social interaction, cooperation, and harmony. Power-oriented people seek influence, authority, and status.
 - * Expectancy theory, which suggests that people are motivated by the expected outcomes of their actions. The outcomes depend on three factors: expectancy, instrumentality, and valence. Expectancy is the belief that one's effort will lead to performance. Instrumentality is the belief that one's performance will lead to rewards. Valence is the value that one assigns to the rewards.
- Organize your team into a suitable structure and size, depending on the nature and complexity of the project. Some of the common types of team structures are:
 - * Functional teams, which are composed of members from the same functional area, such as development, testing, design, etc. They have a clear hierarchy, authority, and communication. They are suitable for stable and predictable projects, but may lack cross-functional collaboration and innovation.
 - * Project teams, which are composed of members from different functional areas, who are dedicated to a specific project. They have a high degree of autonomy, flexibility, and creativity. They are suitable for complex and dynamic projects, but may face conflicts, ambiguity, and resource constraints.
 - * Matrix teams, which are composed of members from different functional areas, who are assigned to multiple projects. They have a dual reporting relationship, to both the functional manager and the project manager. They are suitable for balancing the needs of the organization and the projects, but may experience role confusion, power struggles, and communication issues.
 - Foster a positive and productive team culture, based on shared values, norms, and goals. Some of the ways to do this are:
 - * Define and communicate the vision, mission, and objectives of the project and the team. Align them with the expectations and interests of the stakeholders and the customers.
 - * Establish and enforce the roles and responsibilities of each team member. Clarify the scope, deliverables, deadlines, and quality

standards of the project. Provide the necessary resources, tools, and support for the team to perform their tasks.

- * Promote collaboration, cooperation, and coordination among the team members. Encourage them to share their ideas, opinions, and feedback. Use brainstorming, problem-solving, and decision-making techniques to generate and evaluate solutions. Delegate tasks and authority appropriately, and hold the team accountable for the results.
- * Manage and resolve any conflicts or issues that may arise within or outside the team. Use constructive and respectful communication, negotiation, and mediation skills to address the sources and consequences of the conflicts.

Organizational Behavior in spm

- Organizational behavior (OB) is the study of how people interact and behave within organizations, such as businesses, teams, groups, or communities.
- OB is an interdisciplinary field that draws from psychology, sociology, anthropology, economics, and management.
- OB can help managers and leaders understand, predict, and influence the behavior of individuals and groups in organizations, as well as the performance and outcomes of organizational processes and systems.
- OB covers various topics, such as:
 - Motivation: the factors that drive and sustain human behavior, such as needs, goals, rewards, and incentives.
 - Attitude: the evaluations and beliefs that people hold about themselves, others, situations, or objects, and how they affect behavior and emotions.
 - Perception: the process of selecting, organizing, and interpreting sensory information, and how it influences behavior and judgment.
 - Personality: the stable and unique patterns of traits, characteristics, and preferences that shape an individual's behavior and identity.
 - Emotion: the complex and subjective feelings that arise from internal or external stimuli, and how they affect behavior, cognition, and well-being.
 - Learning: the process of acquiring, modifying, or reinforcing knowledge, skills, abilities, or attitudes through experience, observation, or feedback.
 - Decision making: the process of choosing among alternatives based on preferences, values, criteria, or rules, and how it affects behavior and outcomes.
 - Communication: the process of exchanging information, ideas, or emotions through verbal or nonverbal means, and how it influences

understanding, relationships, and coordination.

- Leadership: the process of influencing, inspiring, or directing others to achieve a common goal or vision, and the traits, styles, and behaviors that characterize effective leaders.
 - Power and politics: the sources, distribution, and use of influence, authority, or control within organizations, and how they affect behavior, conflict, and cooperation.
 - Conflict and negotiation: the process of managing or resolving disagreements, disputes, or differences among individuals or groups, and the strategies, tactics, and outcomes of bargaining or compromising.
 - Culture and diversity: the shared values, norms, beliefs, and practices that shape the identity and behavior of an organization or a group, and the differences and similarities among individuals or groups based on various dimensions, such as gender, race, ethnicity, age, or religion.
 - Ethics and social responsibility: the principles, standards, or codes of conduct that guide the behavior of individuals or organizations, and the obligations or expectations to act in the best interest of society, stakeholders, or the environment.
 - Change and innovation: the process of introducing, adopting, or adapting to new or different ideas, products, processes, or practices within organizations, and the factors that facilitate or hinder change and innovation.
 - Stress and well-being: the physical, psychological, or emotional reactions to demands, challenges, or threats that exceed one's resources or coping abilities, and the factors that promote or impair health, happiness, or satisfaction.
- OB can be applied at different levels of analysis, such as:
 - Individual level: the behavior and outcomes of a single person within an organization, such as motivation, performance, or satisfaction.
 - Group level: the behavior and outcomes of two or more people who interact and share a common identity, goal, or task within an organization, such as teamwork, collaboration, or conflict.
 - Organizational level: the behavior and outcomes of the whole organization or its subunits, such as culture, structure, strategy, or effectiveness.
 - OB can be studied using various methods, such as:
 - Surveys: the collection of data from a sample of individuals or groups using standardized questionnaires or interviews, and the analysis of the data using statistical techniques.
 - Experiments: the manipulation of one or more variables and the measurement of their effects on another variable, while controlling for other factors, and the comparison of the results between different groups or conditions.
 - Case studies: the in-depth examination of a single or multiple in-

stances of a phenomenon, using multiple sources of evidence, such as documents, observations, interviews, or artifacts, and the analysis of the data using qualitative or quantitative techniques.

- Meta-analysis: the systematic review and synthesis of the results of multiple studies on a topic, using statistical techniques to estimate the overall effect size and the sources of variation among the studies.
- OB can be useful for various purposes, such as:
 - Improving individual and organizational performance, productivity, or effectiveness, by identifying and applying the best practices, methods, or interventions.
 - Enhancing individual and organizational well-being, satisfaction, or engagement, by creating and maintaining a positive, supportive, and healthy work environment.
 - Developing individual and organizational capabilities, compet

Best Methods of Staff Selection in spm

Staff selection is the process of choosing the most suitable candidates for a job or a project from a pool of applicants. Staff selection is a crucial step in software project management (spm) as it affects the quality, productivity, and performance of the project team.

Some of the best methods of staff selection in spm are:

- **Assess cognitive ability:** Cognitive ability refers to the mental skills and abilities that are required for problem-solving, learning, reasoning, and decision-making. Cognitive ability tests measure how well a candidate can process information, apply logic, and handle complexity. Cognitive ability tests are useful for selecting staff for spm as they can predict the potential and adaptability of the candidates, as well as their ability to cope with the challenges and changes of software projects.
- **Evaluate learning agility:** Learning agility is the ability and willingness to learn from experience and apply that learning to new and different situations. Learning agility tests measure how well a candidate can learn from feedback, experiment with new approaches, and cope with ambiguity and uncertainty. Learning agility tests are useful for selecting staff for spm as they can indicate the candidates' ability to grow and develop their skills, as well as their ability to handle the dynamic and complex nature of software projects.
- **Situational judgement test (SJT):** SJT is a type of test that presents candidates with realistic scenarios and asks them to choose the most appropriate course of action or response. SJT measures how well a candidate can apply their knowledge, skills, and judgement to specific situations that are relevant to the job or the project. SJT is useful for selecting staff for

spm as it can assess the candidates' practical and interpersonal skills, as well as their fit with the organizational culture and values.

- **Examine samples of previous work:** Samples of previous work are examples of the candidates' past projects, products, or deliverables that demonstrate their skills, abilities, and achievements. Samples of previous work can be portfolios, reports, code, designs, or any other relevant artifacts. Samples of previous work are useful for selecting staff for spm as they can provide evidence of the candidates' quality, creativity, and competence, as well as their suitability for the specific project requirements and expectations.
- **Interview:** Interview is a method of selection that involves a face-to-face or online conversation between the candidates and the interviewers. Interview can be structured, semi-structured, or unstructured, depending on the level of standardization and flexibility of the questions and the answers. Interview can be used to evaluate the candidates' knowledge, skills, personality, motivation, and fit with the job or the project. Interview is useful for selecting staff for spm as it can provide a comprehensive and holistic assessment of the candidates, as well as an opportunity to establish rapport and trust with them.

A mnemonic to remember these five methods of staff selection in spm is:

Assess cognitive ability

Evaluate learning agility

Situational judgement test

Examine samples of previous work

Interview

The first letters of these methods spell out the word **AESEI**, which sounds like "easy". This can help you recall these methods easily.

Motivation in spm

- SPM stands for Software Project Management, which is the discipline of planning, organizing, directing, monitoring and controlling the activities and resources involved in software development projects.
- Motivation is the process of stimulating, directing and sustaining the behavior of people towards a common goal or outcome.
- Motivation in spm is important because it affects the performance, productivity, quality and satisfaction of the software project team members and stakeholders.
- Some of the factors that influence motivation in spm are:

- The nature of the work: The work should be challenging, meaningful, varied and autonomous to foster intrinsic motivation, which is the motivation that comes from within the individual and is driven by interest, enjoyment and personal growth.
 - The work environment: The work environment should be supportive, collaborative, respectful and fair to foster extrinsic motivation, which is the motivation that comes from external rewards and punishments, such as recognition, feedback, incentives and penalties.
 - The individual characteristics: The individual characteristics of the team members, such as their personality, values, attitudes, skills, abilities and needs, should be considered and matched with the work requirements and expectations to foster person-job fit, which is the degree of congruence between the individual and the job.
 - The project characteristics: The project characteristics, such as the scope, complexity, duration, risk and uncertainty, should be clearly defined, communicated and agreed upon by the team members and stakeholders to foster project commitment, which is the degree of attachment and involvement of the team members and stakeholders with the project.
 - The leadership style: The leadership style of the project manager, such as the degree of direction, support, participation and empowerment, should be appropriate and adaptive to the situation and the needs of the team members and stakeholders to foster leader-member exchange, which is the quality of the relationship between the leader and the follower.
- Some of the techniques that can be used to enhance motivation in spm are:
 - Goal setting: Setting specific, measurable, achievable, relevant and time-bound (SMART) goals for the project and the team members, and monitoring and reviewing the progress and performance regularly.
 - Job design: Designing the work tasks and roles to increase the variety, identity, significance, autonomy and feedback (VISAF) of the work, and to balance the skill variety, task identity, task significance, autonomy and feedback (SKITSAF) of the individual.
 - Job enrichment: Adding more responsibility, challenge, growth and recognition to the work tasks and roles to increase the motivation and satisfaction of the team members.
 - Job rotation: Changing the work tasks and roles of the team members periodically to reduce boredom, monotony and fatigue, and to increase learning, development and flexibility.
 - Job sharing: Allowing two or more team members to share the work tasks and roles of a single position to increase collaboration, cooperation and diversity.
 - Reward system: Establishing a fair, transparent and consistent sys-

tem of rewards and recognition for the team members and stakeholders, based on their performance, contribution and achievement.

- Feedback system: Providing timely, specific, constructive and actionable feedback to the team members and stakeholders, based on their performance, contribution and achievement.
- Participation and empowerment: Involving the team members and stakeholders in the decision making and problem solving processes, and delegating authority and responsibility to them, to increase their sense of ownership, autonomy and accountability.
- Team building: Developing and maintaining a cohesive, collaborative and supportive team, based on trust, respect, communication and co-operation, and resolving any conflicts or issues that may arise among the team members and stakeholders.
- Training and development: Providing opportunities and resources for the team members and stakeholders to learn, improve and grow their skills, knowledge and abilities, and to achieve their personal and professional goals.

The Oldham-Hackman Job Characteristic Model in spm

- The Oldham-Hackman Job Characteristic Model, also known as the Job Characteristics Theory or the Core Characteristics Model, is a normative approach to job enrichment created by organizational psychologists J. Richard Hackman and Greg R. Oldham .
- The model focuses on the design of individual tasks rather than the job as a whole, and identifies five core job dimensions that affect the psychological states and outcomes of employees .
- The five core job dimensions are:
 - **Skill variety**: the degree to which the job requires a variety of skills and abilities .
 - **Task identity**: the degree to which the job involves completing a whole and identifiable piece of work .
 - **Task significance**: the degree to which the job has a substantial impact on the lives or work of other people .
 - **Autonomy**: the degree to which the job provides freedom, independence, and discretion to the individual in scheduling the work and determining the procedures to be used .
 - **Feedback**: the degree to which the job provides direct and clear information about the effectiveness of one's performance .
- The model proposes that these five core job dimensions influence three critical psychological states of employees:
 - **Experienced meaningfulness of the work**: the degree to which the individual feels that the work is worthwhile and valuable .
 - **Experienced responsibility for outcomes of the work**: the degree to which the individual feels accountable and in control of the results of the work .

- **Knowledge of the actual results of the work activities:** the degree to which the individual receives clear and timely feedback on the outcomes of the work .
- The model also suggests that these three critical psychological states affect the personal and work outcomes of employees, such as:
 - **Motivation:** the degree to which the individual is internally driven to perform the work .
 - **Performance:** the degree to which the individual meets or exceeds the standards of the work .
 - **Satisfaction:** the degree to which the individual is happy and content with the work .
 - **Absenteeism:** the degree to which the individual is absent from the work .
 - **Turnover:** the degree to which the individual leaves the organization voluntarily or involuntarily .
- The model also acknowledges the role of individual differences in moderating the effects of the core job dimensions on the psychological states and outcomes of employees. The model identifies **growth need strength** as the main individual difference variable that affects how employees respond to enriched jobs .
- Growth need strength refers to the degree to which the individual has a desire to learn, grow, and develop in the work context . Individuals with high growth need strength are more likely to benefit from enriched jobs, while individuals with low growth need strength may prefer more structured and routine

Stress in spm

- SPM stands for **Single Particle Model**, a simplified model of a lithium-ion battery that captures the main dynamics of the electrochemical reactions, the diffusion of lithium ions, and the mechanical stress in the active material.
- Stress in spm refers to the **mechanical stress** generated in the active material particles due to the **volume change** caused by the **intercalation and deintercalation** of lithium ions.
- Stress in spm is an important factor that affects the **performance, safety, and lifetime** of lithium-ion batteries, as it can lead to **particle fracture, electrode cracking, and capacity fade**.
- Stress in spm can be modeled by **coupling** the spm with a **stress sub-model** that accounts for the **elastic and plastic deformation** of the particles, the **interaction between particles**, and the **boundary conditions** of the electrode.
- Stress in spm can be **measured** by using **sensors** such as **strain gauges** or **acoustic emission** devices, or by using **non-invasive techniques** such as **X-ray diffraction** or **Raman spectroscopy**.
- Stress in spm can be **reduced** by using **materials** with **low volume**

change, high fracture toughness, and good conductivity, or by using **design strategies** such as **coating, porosity control, and particle size distribution**.

Some possible mnemonics and learning tricks for stress in spm are:

- To remember the definition of spm, think of **Single Particle Model** as a **Simple Physical Model** of a battery.
- To remember the causes of stress in spm, think of **Volume** change, **Intercalation**, and **Deintercalation** as **VIDeo** games that can cause stress.
- To remember the effects of stress in spm, think of **P**article fracture, **E**lectrode cracking, and **C**apacity fade as **PEC**uliar problems of batteries.
- To remember the components of the stress sub-model, think of **E**lastic and **P**lastic deformation, **I**nteraction between particles, and **B**oundary conditions as **EPIC** Battles in a game.
- To remember the methods of measuring stress in spm, think of **S**train gauges, **A**coustic emission, **X**-ray diffraction, and **R**aman spectroscopy as **SAXoR** instruments that can measure sound and stress.
- To remember the ways of reducing stress in spm, think of **M**aterials, **C**oating, **P**orosity control, and **P**article size distribution as **MCP**P, a rap group that can reduce stress with music.

Health and Safety in spm

- Health and safety in spm refers to the prevention and management of occupational health and safety risks in the field of scanning probe microscopy (spm), which is a technique that uses a physical probe to scan the surface of a sample at the nanoscale.
- Some of the potential hazards and risks associated with spm include:
 - Exposure to high voltage, laser, or microwave radiation from the spm equipment or the sample.
 - Contact with sharp or fragile probes, needles, or tips that can cause cuts, punctures, or infections.
 - Handling of toxic, corrosive, flammable, or explosive materials or substances that may be used as samples or solvents.
 - Generation of static electricity, sparks, or heat that can ignite or damage the spm equipment or the sample.
 - Mechanical or electrical failure of the spm equipment or the sample stage that can result in injury or damage.
 - Ergonomic or psychological stress from working in a confined, dark, or noisy environment or from performing repetitive or complex tasks.
- Some of the best practices and guidelines for ensuring health and safety in spm include:
 - Following the standard operating procedures (SOPs) and the manufacturer’s instructions for the spm equipment and the sample preparation and handling.
 - Wearing appropriate personal protective equipment (PPE) such as

gloves, goggles, lab coats, or masks when working with the spm equipment or the sample.

- Using proper tools and techniques for mounting, aligning, scanning, and removing the probes, needles, or tips and for cleaning and disposing of them.
- Avoiding direct contact with the spm equipment or the sample when they are energized, heated, or cooled and using caution when adjusting or moving them.
- Keeping the spm equipment and the sample area clean, organized, and free of clutter and ensuring adequate ventilation and lighting.
- Reporting any accidents, incidents, or malfunctions to the supervisor or the spm operator and seeking medical attention if needed.
- Taking regular breaks and stretching exercises to prevent fatigue, strain, or injury and seeking help or support if feeling stressed or overwhelmed.

Ethical and Professional Concerns in spm

- spm stands for Software Project Management, which is the discipline of planning, organizing, leading, and controlling software projects.
- Ethical and professional concerns in spm are the issues that arise from the moral and legal obligations of software project managers and team members, as well as the impact of their decisions and actions on the stakeholders, society, and environment.
- Some of the ethical and professional concerns in spm are:
 - **Quality and reliability:** Software project managers and team members should ensure that the software products they deliver meet the requirements and expectations of the customers and users, and are free from defects and errors that could cause harm or loss. They should also follow the standards and best practices of software engineering, and use appropriate methods and tools to ensure quality and reliability.
 - **Intellectual property rights:** Software project managers and team members should respect the intellectual property rights of others, such as patents, copyrights, trademarks, and trade secrets, and avoid plagiarism, infringement, or misappropriation. They should also protect their own intellectual property rights, and disclose any conflicts of interest or potential risks to the stakeholders.
 - **Privacy and security:** Software project managers and team members should protect the privacy and security of the data and information that they collect, store, process, or transmit, and comply with the relevant laws and regulations. They should also design and implement software products that are secure from unauthorized access, modification, or misuse, and inform the stakeholders of any vulnera-

bilities or breaches.

- **Social and environmental responsibility:** Software project managers and team members should consider the social and environmental implications of their software products, and avoid or minimize any negative impacts on the society or the environment. They should also promote the positive impacts, such as enhancing the quality of life, advancing the knowledge, or solving the problems of the society or the environment.
- **Professional conduct and competence:** Software project managers and team members should adhere to the code of ethics and professional conduct of their profession, such as the IEEE Code of Ethics or the ACM Code of Ethics and Professional Conduct. They should also maintain and improve their professional competence, and seek feedback and guidance from their peers, mentors, or experts. They should also act with honesty, integrity, fairness, and respect, and avoid any misconduct, such as fraud, deception, discrimination, or harassment.

Working in Teams in spm

- SPM stands for Software Project Management, which is the discipline of planning, organizing, leading, and controlling the work of a team to achieve specific goals and meet specific success criteria.
- Working in teams is an essential skill for software project managers, as they need to coordinate the efforts of multiple stakeholders, such as developers, testers, clients, users, and sponsors.
- Some of the benefits of working in teams are:
 - Increased productivity and quality, as team members can share their expertise, skills, and resources, and provide feedback and support to each other.
 - Enhanced creativity and innovation, as team members can generate and evaluate more ideas, and combine different perspectives and approaches.
 - Improved communication and collaboration, as team members can establish trust, rapport, and mutual understanding, and resolve conflicts and issues effectively.
 - Higher motivation and satisfaction, as team members can feel a sense of belonging, recognition, and achievement, and enjoy the social and emotional aspects of teamwork.
- Some of the challenges of working in teams are:
 - Coordination and integration, as team members need to align their goals, roles, tasks, and schedules, and ensure consistency and compatibility of their work outputs.
 - Diversity and complexity, as team members may have different backgrounds, personalities, preferences, and expectations, and face various technical and organizational issues.

- Conflict and competition, as team members may have conflicting or competing interests, opinions, or agendas, and experience power struggles, disagreements, or misunderstandings.
- Performance and accountability, as team members need to monitor and evaluate their own and others' work, and provide and receive constructive feedback and criticism.
- Some of the best practices for working in teams are:
 - Define and communicate the team vision, mission, objectives, and scope, and ensure they are aligned with the project and organizational goals and strategies.
 - Select and assign the team members based on their competencies, availability, and suitability, and define their roles and responsibilities clearly and explicitly.
 - Establish and follow the team norms, rules, and procedures, and agree on the communication and collaboration methods, tools, and protocols.
 - Plan and schedule the team activities, tasks, and deliverables, and allocate the resources and budget accordingly, and track and report the team progress and performance regularly.
 - Encourage and facilitate the team participation, contribution, and interaction, and foster a positive and supportive team culture and climate.
 - Manage and resolve the team conflicts and problems, and address the team issues and concerns promptly and effectively.
 - Recognize and reward the team achievements and successes, and celebrate the team milestones and accomplishments.

Decision Making in spm

- Decision making in software product management (SPM) is the process of choosing the best course of action among various alternatives to achieve the desired outcomes for a software product.
- Decision making in SPM is influenced by several factors, such as the product strategy, the business model, the product lifecycle, the market needs, the stakeholder expectations, the technical solutions, the available resources, and the risks and uncertainties.
- Decision making in SPM can be done at three levels: strategic, tactical, and operational.
 - At the strategic level, decisions are mostly related to designing a product strategy and defining the overall business model. For example, deciding what product vision, value proposition, target market, and competitive advantage to pursue.
 - At the tactical level, decisions are mostly related to guiding the product management processes, such as release planning, lifecycle management, and roadmapping. For example, deciding what features, quality attributes, and dependencies to include in each release, how

- to prioritize and allocate resources, and how to align the product roadmap with the product strategy and the market needs.
 - At the operational level, decisions are mostly related to determining the technical solutions for product architecture, required infrastructure and deployment, and schedules operations. For example, deciding what technologies, tools, methods, and standards to use, how to design, test, and deploy the product, and how to monitor and maintain the product performance and quality.
- Decision making in SPM can also be classified based on whether it is done by a group or an individual, and whether it is programmed or non-programmed.
 - Group decision making involves the participation and collaboration of multiple stakeholders, such as product managers, developers, customers, users, and executives. Group decision making can have advantages, such as increased creativity, diversity, and acceptance, but also challenges, such as conflicts, biases, and delays.
 - Individual decision making involves the sole responsibility and authority of one person, usually the product manager. Individual decision making can have advantages, such as speed, consistency, and accountability, but also challenges, such as limited information, perspectives, and feedback.
 - Programmed decision making involves the use of predefined rules, procedures, and criteria to make routine and repetitive decisions. Programmed decision making can have advantages, such as efficiency, accuracy, and reliability, but also challenges, such as rigidity, inflexibility, and obsolescence.
 - Non-programmed decision making involves the use of intuition, judgment, and creativity to make novel and complex decisions. Non-programmed decision making can have advantages, such as adaptability, innovation, and differentiation, but also challenges, such as uncertainty, ambiguity, and error.
- Decision making in SPM can be improved by using various techniques, such as evidence-based management, decision analysis, decision support systems, and decision making frameworks.
 - Evidence-based management is the practice of making decisions based on the best available evidence from multiple sources, such as data, research, experience, and stakeholder feedback. Evidence-based management can help reduce biases, assumptions, and uncertainties, and increase the validity, reliability, and transparency of decisions.
 - Decision analysis is the process of applying quantitative and qualitative methods to evaluate the alternatives, criteria, and outcomes of decisions. Decision analysis can help structure, model, and compare the decision problems, and identify the optimal or satisfactory solutions.
 - Decision support systems are the tools and systems that provide infor-

mation, analysis, and recommendations to support decision making. Decision support systems can help automate, simplify, and enhance the decision making processes, and provide interactive and intelligent features, such as visualization, simulation, and optimization.

- Decision making frameworks are the sets of principles, guidelines, and best practices that help organize, standardize, and streamline the decision making processes. Decision making frameworks can help define the roles, responsibilities, and steps of decision making, and ensure the alignment, consistency, and quality of decisions.

Organizational Structures in spm

Organizational structures in software project management (spm) are the ways of organizing the roles, responsibilities, and relationships of the project team members. Different organizational structures have different advantages and disadvantages for the project, depending on the project size, complexity, scope, and objectives. Some of the common organizational structures in spm are:

- **Functional structure:** In this structure, the project team members are grouped by their functional areas, such as engineering, marketing, finance, etc. Each functional group reports to a functional manager, who is responsible for the coordination and supervision of the group. The functional manager also acts as the project manager for the projects within his or her domain. The advantages of this structure are that it allows for specialization, efficiency, and clear lines of authority. The disadvantages are that it may cause communication problems, conflicts, and delays across different functional groups, and that it may not be suitable for complex or multidisciplinary projects.
- **Projectized structure:** In this structure, the project team members are grouped by their projects, rather than their functional areas. Each project has a dedicated project manager, who has full authority and responsibility over the project and the project team. The project manager reports to a senior manager or a project management office (PMO), who oversees multiple projects. The advantages of this structure are that it allows for flexibility, responsiveness, and customer focus. The disadvantages are that it may cause duplication of resources, lack of continuity, and low morale among the project team members, and that it may not be suitable for stable or routine projects.
- **Matrix structure:** In this structure, the project team members are grouped by both their functional areas and their projects. Each project team member reports to both a functional manager and a project manager, who share the authority and responsibility over the project and the project team. The functional manager is responsible for the allocation and evaluation of the project team members, while the project manager is responsible for the planning and execution of the project. The advantages

of this structure are that it allows for balance, integration, and innovation. The disadvantages are that it may cause role ambiguity, power struggles, and stress among the project team members, and that it may not be suitable for simple or independent projects.

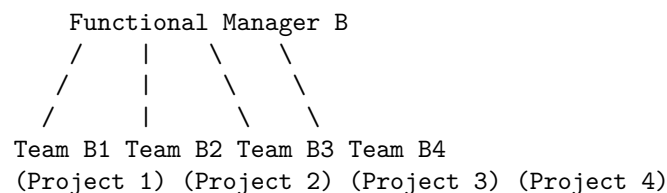
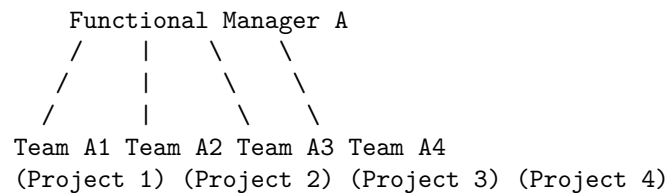
- **Hybrid structure:** In this structure, the project team members are grouped by a combination of different structures, depending on the needs and characteristics of the project. For example, a project may have a functional structure for some phases, a projectized structure for some phases, and a matrix structure for some phases. The advantages of this structure are that it allows for customization, adaptation, and optimization. The disadvantages are that it may cause complexity, inconsistency, and confusion among the project team members, and that it may not be suitable for standard or predictable projects.

A mnemonic to remember the four types of organizational structures in spm is:

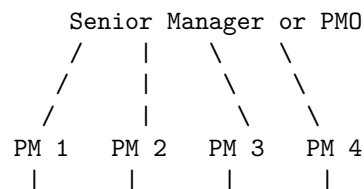
- **F**unctional structure: **F**unctional groups
- **P**rojectized structure: **P**roject groups
- **M**atrix structure: **M**ixed groups
- **H**ybrid structure: **H**eterogeneous groups

A diagram to illustrate the four types of organizational structures in spm is:

Functional structure:



Projectized structure:



channels.

- They may encounter coordination and collaboration issues due to the complexity and interdependence of software tasks, the variability of team member availability and responsiveness, and the ambiguity of roles and responsibilities.
- They may suffer from social and emotional isolation, stress, and burnout due to the lack of social support, feedback, and recognition from peers and managers.
- They may face cultural and ethical conflicts due to the differences in values, norms, and expectations among team members, customers, and stakeholders.
- Some of the best practices for managing dispersed and virtual teams in spm are:
 - Establish a clear and shared vision, mission, and objectives for the project, and communicate them frequently and consistently to all team members.
 - Define and document the roles, responsibilities, and expectations of each team member, and assign tasks based on their skills, preferences, and availability.
 - Use a combination of synchronous and asynchronous communication tools and methods, such as video conferencing, instant messaging, email, and wikis, to facilitate information sharing, problem solving, and decision making.
 - Create and maintain a team charter, a code of conduct, and a conflict resolution mechanism to foster trust, respect, and cooperation among team members.
 - Provide regular and constructive feedback, recognition, and rewards to team members, and celebrate their achievements and milestones.
 - Encourage and support team members to interact socially and personally, and to build rapport and camaraderie.
 - Monitor and evaluate the performance and progress of the team and the project, and make adjustments and improvements as needed.

Communications Genres in spm

- Communications genres are the types of documents or artifacts that are used to communicate information in software project management (spm).
- Communications genres can be classified into four categories: **planning**, **reporting**, **controlling**, and **supporting**.
- Planning genres are those that help define the scope, objectives, schedule, budget, risks, and quality of the project. Examples of planning genres are project charter, project plan, work breakdown structure, risk register, quality plan, etc.
- Reporting genres are those that provide information on the status,

progress, performance, and outcomes of the project. Examples of reporting genres are status report, progress report, performance report, deliverable, etc.

- Controlling genres are those that help monitor and control the project activities, resources, and changes. Examples of controlling genres are issue log, change request, change log, variance analysis, corrective action plan, etc.
- Supporting genres are those that facilitate the communication and collaboration among the project stakeholders, such as team members, customers, sponsors, and suppliers. Examples of supporting genres are meeting agenda, meeting minutes, email, memo, presentation, etc.
- A mnemonic to remember the four categories of communications genres is **PRCS** (Planning, Reporting, Controlling, Supporting).
- A learning trick to understand the purpose and audience of each communications genre is to ask the following questions:
 - **What** is the information that needs to be communicated?
 - **Why** is the information important or relevant for the project?
 - **Who** are the intended recipients or users of the information?
 - **How** will the information be delivered or accessed?
 - **When** will the information be available or updated?

Communication Plans in SPM

- SPM stands for Strategic Performance Management, which is a multistep process that guides an organization's leadership in designing and revising a system of strategic performance management.
- Communication is a critical component of the SPM process, as it helps to engage and collaborate with external and internal stakeholders, and to communicate the vision, goals, and progress of the organization.
- A communication plan is an outline of how to communicate important, ongoing information to key stakeholders in the SPM process. It helps to ensure that everyone is on the same page, and that the communication is consistent, clear, and timely.
- A communication plan typically includes the following elements:
 - **Purpose:** The reason and objectives of the communication plan, such as informing, persuading, or motivating stakeholders.
 - **Audience:** The groups or individuals who need to receive the communication, such as employees, customers, partners, or media.
 - **Message:** The key points and information that need to be communicated, such as the vision, goals, achievements, or challenges of the organization.
 - **Channel:** The medium or method of communication, such as email, newsletter, website, social media, or meeting.

- **Frequency:** The schedule or timing of the communication, such as daily, weekly, monthly, or quarterly.
- **Responsibility:** The person or team who is in charge of creating and delivering the communication, such as the project manager, the communication manager, or the executive director.
- **Feedback:** The mechanism or process of collecting and analyzing the responses and reactions of the audience, such as surveys, polls, comments, or reviews.
- A communication plan can be presented in a table or a matrix format, with each element as a column or a row, and each communication activity as a cell. For example:

Purpose	Audience	Message	Channel	Frequency	Responsibility	Feedback
Inform	Employees	Vision and goals of the organization	Email	Quarterly	Executive director	Survey
Persuade	Customers	Benefits and features of the new product	Website	Monthly	Marketing manager	Comments
Motivate	Partners	Success stories and best practices	Newsletter	Weekly	Project manager	Reviews

- A communication plan can help to improve the SPM process by:
 - Aligning the stakeholders with the vision and goals of the organization
 - Building trust and credibility among the stakeholders
 - Enhancing the collaboration and coordination among the stakeholders
 - Increasing the awareness and understanding of the organization's performance
 - Encouraging the feedback and input from the stakeholders
 - Celebrating the achievements and recognizing the contributions of the stakeholders
- A communication plan can be evaluated and revised based on the feedback and the changing needs of the organization and the stakeholders. A communication plan should be flexible and adaptable to the dynamic environment and the evolving SPM process.

Leadership in spm

- Leadership is the process of influencing and motivating others to achieve a common goal or vision.
- Leadership is essential for software project management (spm) because software projects are complex, uncertain, and dynamic, and require coordination and collaboration among various stakeholders.
- Leadership styles are the ways that leaders behave and communicate with their followers. Different leadership styles may be more or less effective depending on the situation and the characteristics of the followers.
- Some of the common leadership styles are:
 - Autocratic: The leader makes all the decisions and gives orders to the followers. The leader does not seek input or feedback from the followers. This style may be effective when the leader has high expertise and the followers are inexperienced or need close supervision. However, this style may also cause resentment, low morale, and high turnover among the followers.
 - Democratic: The leader involves the followers in the decision-making process and encourages participation and feedback. The leader respects and values the opinions and ideas of the followers. This style may be effective when the leader wants to build trust, commitment, and creativity among the followers. However, this style may also cause delays, conflicts, and confusion if the leader does not provide clear guidance and direction.
 - Laissez-faire: The leader delegates most of the authority and responsibility to the followers and gives them freedom and autonomy to complete their tasks. The leader provides minimal supervision and intervention. This style may be effective when the followers are highly skilled, motivated, and self-reliant. However, this style may also cause chaos, lack of accountability, and poor performance if the leader does not monitor and support the followers.
 - Transformational: The leader inspires and motivates the followers to achieve a shared vision and higher goals. The leader provides a clear direction, communicates a compelling message, and models the desired behaviors. The leader also empowers and develops the followers by providing them with coaching, feedback, and recognition. This style may be effective when the leader wants to foster innovation, change, and excellence among the followers. However, this style may also cause unrealistic expectations, burnout, and dependency if the leader does not balance the demands and resources of the followers.
- A mnemonic to remember the four leadership styles is **ADLT** (Autocratic, Democratic, Laissez-faire, Transformational).
- Some of the skills and competencies that a software project manager (spm)

needs to be an effective leader are:

- Vision: The ability to create and communicate a clear and compelling vision of the desired outcome and direction of the software project.
- Communication: The ability to convey information and ideas effectively and efficiently to various stakeholders using appropriate verbal, written, and visual methods.
- Negotiation: The ability to reach mutually beneficial agreements and resolve conflicts among different parties with diverse interests and expectations.
- Teamwork: The ability to collaborate and cooperate with others to achieve common goals and objectives.
- Motivation: The ability to stimulate and sustain the interest, enthusiasm, and commitment of the project team and other stakeholders.
- Empathy: The ability to understand and appreciate the feelings, thoughts, and perspectives of others and to respond accordingly.
- Ethics: The ability to act with honesty, integrity, and professionalism and to adhere to the relevant standards and principles of the software engineering profession.